Cheatsheet · 34 chapters on one page

Java cheatsheet

Every formula, key idea, table, memory aid and common mistake from the chapters, gathered in one place for quick revision. Nothing here is new. Each card links back to the lesson that teaches it, so open the lesson when a line does not make sense yet.

316 key ideas | 183 formulas | 116 tables | 178 tips and traps

STEP 1 Complete Java Session

1.1 How Java Runs: Classes, Variables and Data Types

Before writing bigger programs, it helps to know what actually happens when you press Run. This chapter follows a Java file from source code to output, then covers the building blocks every program uses: classes, objects, variables, data types, casting, static members, the main method and access modifiers. Each idea is shown in Java with the Python and JavaScript equivalent beside it.

KEY IDEAS

- **From source code to output** A Java program goes through two stages. The compiler (javac) turns your .java source into bytecode in a .class file.
- **Classes and objects** Every Java program lives inside a class : a blueprint that groups data (variables) and behaviour (methods). An object is a real thing built from that blueprint, with its own state.
- **Data types, sizes and default values** Java has eight primitive types that hold the value directly, and non-primitive (reference) types such as String , arrays and every class, which hold a reference.
- **Variables and identifiers** Identifier rules: only letters, digits, _ and $, must not start with a digit, and must not be one of Java's keywords (all keywords are lower case, such as class , int , static). Identifiers are case sensitive.
- **Widening, narrowing and overflow** Arithmetic on byte , short and char is done in int . So this does not compile:
- **Static and non-static members** A static member belongs to the class and exists once. A non-static (instance) member belongs to each object. A static method such as main has no object, so it cannot use instance members directly:
- **The main method**
- **Access modifiers**

FORMULAS

Data types, sizes and default values

Exam favourite

$$-2^{b-1}$$

Lesson: Data types, sizes and default values

Data types, sizes and default values

$$2^{b-1} - 1$$

Lesson: Data types, sizes and default values

Narrowing (explicit) Exam favourite

$$200 - 256 = -56$$

Lesson: Widening, narrowing and overflow

TABLES

Table: part · what it contains · who needs it

Part	What it contains	Who needs it
JVM	Class loader, bytecode verifier, interpreter, JIT compiler, garbage collector	Runs bytecode
JRE	JVM plus the standard class libraries	Anyone who only runs Java programs
JDK	JRE plus development tools: `javac`, `java`, `jar`, `javadoc`, debugger	Developers and testers who write code

Lesson: From source code to output

Table: piece · meaning

Piece	Meaning
`new`	A keyword that creates a new object every time it runs
`ClassName()`	The constructor that initialises the new object
`ref`	A reference variable that stores the address of the object, not the object itself

Lesson: Classes and objects

Table: type · size · range · default

Type	Size	Range	Default	Example
boolean	JVM dependent	true, false	false	`boolean ok = true;`
char	2 bytes	'\u0000' to '□' (0 to 65,535)	'\u0000'	`char c = 'A';`
byte	1 byte	−128 to 127	0	`byte b = 10;`
short	2 bytes	−32,768 to 32,767	0	`short s = 1000;`
int	4 bytes	about ±2.1 billion	0	`int n = 100000;`
long	8 bytes	about $\pm 9.2 \times 10^{18}$	0L	`long big = 9000000000L;`
float	4 bytes	about 7 significant digits	0.0f	`float f = 3.14f;`
double	8 bytes	about 15 significant digits	0.0	`double d = 3.14;`

Lesson: Data types, sizes and default values

Table: · local variable · global variable (instance or static)

	Local variable	Global variable (instance or static)
Declared	Inside a method	Inside the class, outside every method
Visible	Only inside that method	Anywhere in the class, and outside it depending on the access modifier
Must be initialised before use?	Yes, or compilation error	No, it gets the default value

Lesson: Variables and identifiers

Table: part of public static void main(string[] args) · why

Part of `public static void main(String[] args)`	Why
public	The JVM must be able to see and call it from outside the class
static	It runs before any object exists, so it must belong to the class
void	It returns nothing to the JVM (use `System.exit(code)` for an exit code)
main	The name the JVM looks for as the starting point
String[] args	Command line arguments, for example `java App test 3` gives `args[0] = "test"`

Lesson: The main method

Table: modifier · same class · same package · subclass in another package

Modifier	Same class	Same package	Subclass in another package	Anywhere
public	Yes	Yes	Yes	Yes
protected	Yes	Yes	Yes	No
default (no keyword)	Yes	Yes	No	No
private	Yes	No	No	No

Lesson: Access modifiers

TIPS AND TRAPS

REMEMBER — The same `.class` file runs on Windows, macOS and Linux because each has its own JVM. This is what "write once, run anywhere" means: the bytecode is platform independent, the JVM is not.

Lesson: From source code to output

REMEMBER — Python also compiles to bytecode (`.pyc`) and runs it in a virtual machine. JavaScript engines such as V8 parse the source and JIT compile it. Java makes the compile step visible and separate.

Lesson: From source code to output

COMMON MISTAKE — The default char is the null character `'\u0000'`, not a space. It prints as nothing visible, which is why it is often mistaken for a space.

Lesson: Data types, sizes and default values

REMEMBER — Static members are loaded when the class is loaded. Use them for things that truly belong to the class, such as a counter or a constant, not as a shortcut to avoid creating objects.

Lesson: Static and non-static members

IN THE EXAM — Can a Java program run without a main method? A standalone application cannot: main is the entry point. Frameworks such as TestNG or JUnit provide their own main and call your annotated methods instead.

Lesson: The main method

REMEMBER — Python has no enforced access modifiers: a leading underscore (`_name`) is a convention, and a double underscore triggers name mangling. JavaScript classes have truly private fields written `#name`.

Lesson: Access modifiers

1.2 Operators, Conditions, Loops and Patterns

Programs make decisions and repeat work. This chapter covers the operators that build conditions, the blocks that choose a path, and the loops that repeat it, then puts them together in the loop and pattern programs from the session, including two prime checks from the notes that hide a boundary value bug.

KEY IDEAS

- **Operators and precedence**
- **if, else if, nested if and switch** Conditions must be boolean expressions built from relational and logical operators. Java does not treat 0 or an empty string as false, unlike Python and JavaScript.
- **while, do while, for and for each** All three parts of a for are optional: for (; i < 5;) works if i is set before and changed inside. Leave out the condition and the loop never ends.
- **break, continue and labelled loops**
- **Classic loop programs, and a prime bug** The session's three prime checks get faster: count every factor (O(n)), stop at n/2, stop at √n. But they hide boundary bugs : none of them rejects 0 and 1, the n/2 version calls 4 prime, and the √n version calls 9 and 25 prime.
- **Pattern programs with nested loops** A pattern is a grid: the outer loop chooses the row , the inner loop prints the columns of that row. The trick is the inner loop's range as a function of the row number.

FORMULAS

Pattern programs with nested loops

Exam favourite

$$n \times n$$

Lesson: Pattern programs with nested loops

Pattern programs with nested loops

$$\frac{n(n+1)}{2}$$

Lesson: Pattern programs with nested loops

TABLES

Table: group · operators · example

Group	Operators	Example
Arithmetic	`+ - * / %`	`17 / 5 = 3, 17 % 5 = 2`
Relational	`== != > < >= <=`	`a >= 18` gives true or false
Logical	`&& || !`	`age > 18 && hasId`
Assignment	`= += -= *= /= %=`	`total += price`
Unary	`++ -- + - !`	`i++, ++i`
Ternary	`condition ? a : b`	`n % 2 == 0 ? "even" : "odd"`
Bitwise and shift	`& | ^ ~ << >> >>>`	`5 & 3 = 1, 1 << 4 = 16`

Lesson: Operators and precedence

Table: loop · checks the condition · use it when

Loop	Checks the condition	Use it when
`while (cond) { }`	Before each pass	You do not know how many times, for example "until the number becomes 0"
`do { } while (cond);`	After each pass, so it runs at least once	A menu that must show at least once, or retrying an input
`for (init; cond; update) { }`	Before each pass	Counting a known number of times
`for (int x : arr) { }`	Once per element	Reading every element of an array or collection

Lesson: while, do while, for and for each

Table: pattern · row r prints · inner loop

Pattern	Row r prints	Inner loop
Square	n stars	`c = 1 … n`
Right triangle	r stars	`c = 1 … r`
Inverted triangle	n – r + 1 stars	`c = r … n` or count down
Pyramid	n – r spaces, then 2r – 1 stars	two inner loops

Lesson: Pattern programs with nested loops

TIPS AND TRAPS

COMMON MISTAKE Java's `%` keeps the sign of the left operand: `-7 % 3 = -1`. Python's `%` follows the right operand: `-7 % 3 = 2`. Programs that check "is odd" with `n % 2 == 1` fail for negative numbers in Java. Use `n % 2 != 0`.

Lesson: Operators and precedence

REMEMBER Python has `while` and `for x in ...` (a for each loop), with `range(start, stop, step)` for counting. It has no do while. JavaScript has all four, plus `for...of` for values and `for...in` for keys.

Lesson: while, do while, for and for each

REMEMBER Python has no labels. Leave nested loops with a flag, a function that returns, or the `for ... else` clause. JavaScript supports labels like Java.

Lesson: break, continue and labelled loops

IN THE EXAM This is boundary value analysis in practice. The edges here are 0, 1, 2, small composites such as 4, and perfect squares such as 9 and 25, exactly where < versus <= matters.

Lesson: Classic loop programs, and a prime bug

REMEMBER Two nested loops of n each run

$$n \times n$$

times, so printing a square is O(n²). The triangle prints

$$\frac{n(n+1)}{2}$$

stars, still O(n²).

Lesson: Pattern programs with nested loops

1.3 Methods and Constructors

The session builds methods step by step, from one Addition program to the next, and then does the same with eleven constructor programs. This chapter follows that path: where a method lives, how it is called, what it receives and returns, and how a constructor prepares a new object, including the four constructor chaining cases that decide which fields end up with which values.

KEY IDEAS

- **Why methods** A method is a named block of code that performs one task. Writing the code once and calling it many times removes repetition, makes the program easier to read, and means a fix in one place fixes every call.
- **Calling static and instance methods**
- **Parameters, return values and pass by value** Parameters are the variables in the signature, arguments are the values passed in the call. A method declared void returns nothing, any other return type must return a value of that type (Addition5 prints, Addition6 returns).

- **Method overloading** Several methods may share a name if their parameter lists differ in number, type or order. The compiler picks the best match at compile time. A different return type alone is not enough.
- **The call stack and recursion** Every call pushes a frame holding its parameters and local variables onto the call stack. Returning pops it. A method that calls itself is recursive , and it needs a base case, or the stack overflows with StackOverflowError .
- **Constructors**
- **this and constructor chaining** this refers to the current object. this.i = i separates the field from a parameter with the same name. this(...) calls another constructor of the same class and must be the first statement .

TABLES

Table: method · how to call it · note

Method	How to call it	Note
static, same class	`add();`	Direct call works inside the class
static, any class	`Add.add();`	Recommended: shows the method belongs to the class
static, through an object	`a.add();`	Compiles, but is misleading and not recommended
instance	`Addition2 a = new Addition2();` `a.add();`	Needs an object, because it uses that object's fields

Lesson: Calling static and instance methods

Table: rule · detail

Rule	Detail
Name	Exactly the class name
Return type	None, not even void. Adding void makes it an ordinary method
Purpose	Initialise the fields of a new object
Default constructor	If a class declares no constructor at all, the compiler adds an empty one with no arguments
Once you write one	The compiler adds nothing. Calling `new Cons()` when only `Cons(int)` exists is a compilation error
private	Allowed, but then other classes cannot create objects (used for singletons and utility classes)

Lesson: Constructors

TIPS AND TRAPS

REMEMBER

Java always passes **by value**. For a primitive the value is the number, so the caller's variable never changes. For an object the value is the reference, so the method can change the object's fields, but assigning a new object to the parameter does not affect the caller. Python and JavaScript behave the same way.

Lesson: Parameters, return values and pass by value

REMEMBER

Python and JavaScript have no overloading: a second definition replaces the first. They use default parameter values or check the arguments inside one function instead.

Lesson: Method overloading

IN THE EXAM

Printing an object without `toString()` shows something like `Cons9@1b6d3586`: the class name and a hash code, not the memory address itself. `c == c1` compares references, so two separate objects are never == even if their fields match.

Lesson: this and constructor chaining

1.4 OOP in Practice

The session builds OOP around one family of classes: Mobile, SmartMobile, IPhone and Samsung. This chapter keeps those classes, so every idea, from extends to downcasting, lands on code you have already met. Each section ends with what compiles, what runs, and what fails at run time.

KEY IDEAS

- **The four pillars** The word polymorphism joins the Greek poly (many) and morph (form). In code, it means the same call can behave differently depending on the arguments or on the object.
- **Overloading and method mapping** Overloading is compile time polymorphism : the compiler fixes which method runs. When no overload matches the argument types exactly, it widens the arguments and picks the most specific overload that fits.
- **Inheritance and its types** A child class extends a parent and gets its fields and methods without rewriting them. Every class without an extends silently extends Object . Inheritance models an is-a relationship: an IPhone is a SmartMobile, which is a Mobile.
- **Method overriding** Overriding gives a child its own version of a parent method with the same name and the same parameters . Which version runs is decided at run time by the actual object, so it is called runtime polymorphism or dynamic binding.
- **super and the order of construction** super refers to the parent part of the current object. super.method() calls the parent version of an overridden method, super.field reads a hidden parent field, and super(...) picks a parent constructor.
- **Upcasting, downcasting and instanceof** The session's DownCasting program writes (Iphone5) new Mobile5() . It compiles, then fails at run time, because a plain Mobile5 object never becomes an iPhone.
- **Abstraction with abstract classes**
- **Interfaces** An interface is a contract: a list of methods a class promises to provide. A class can implement several interfaces, which is how Java offers multiple inheritance safely.
- **Encapsulation with getters and setters** Encapsulation wraps data and the code that changes it in one class, and hides the data with private .

TABLES

Table: pillar · idea · java tools

Pillar	Idea	Java tools
Polymorphism	One name, many forms	Overloading (compile time), overriding (run time)
Inheritance	A child class reuses and extends a parent class	`extends`, `implements`, `super`
Abstraction	Show what an object does, hide how it does it	`abstract` classes and methods, interfaces
Encapsulation	Keep data private and control every change	`private` fields, getters and setters

Lesson: The four pillars

Table: parent member · inherited by a child in the same package · child in another package

Parent member	Inherited by a child in the same package	Child in another package
public	Yes	Yes
protected	Yes	Yes
default	Yes	No
private	No	No

Lesson: Inheritance and its types

Table: rule · detail

Rule	Detail
Signature	Same name and parameter list. A different list is overloading, not overriding
Return type	Same, or a subtype (covariant return)
Access	Same or wider. A public method cannot become protected in the child
Cannot override	`static` (hidden instead), `final` and `private` methods
`@Override`	Optional, but the compiler then checks that a parent method really exists. A typo such as `0s()` becomes an error instead of a silent new method

Lesson: Method overriding

Table: fact · detail

Fact	Detail
Abstract method	Signature only, ending in a semicolon, marked `abstract`
Abstract class	Any class with at least one abstract method must be declared `abstract`. It may also have no abstract methods at all
Objects	`new Mobile4()` is a compilation error. Use a child object, directly or through a parent reference
What it can hold	Fields, constructors, concrete methods and static methods, so it gives partial abstraction
The child	Must override every abstract method, or be declared abstract itself

Lesson: Abstraction with abstract classes

Table: member · what java assumes

Member	What Java assumes
Method without a body	`public abstract`, so implementing methods must be `public`
Variable	`public static final`, a constant that must be initialised
`default` method (JDK 8)	Has a body and is inherited. A class may override it
`static` method (JDK 8)	Called as `Mobile6.test1()` only, never through an object
Objects	`new Mobile6()` is a compilation error

Lesson: Interfaces

TIPS AND TRAPS

COMMON MISTAKE If two overloads fit equally well, for example `sum(int, long)` and `sum(long, int)` called with `sum(1, 2)`, the call is **ambiguous** and does not compile.

Lesson: Overloading and method mapping

REMEMBER A class can extend only **one** class. `class C1 extends P1, P2` is a compilation error. If both parents had a method with the same name, the child would not know which to inherit: the **diamond problem**. Java allows multiple inheritance only through interfaces.

Lesson: Inheritance and its types

REMEMBER Every constructor starts by running a parent constructor. If you do not write `super(…)` or `this(…)` as the first line, the compiler inserts `super()`. So building an IPhone15 runs Phone, then SmartPhone, then IPhone15. If the parent has no no-argument constructor, the child must call `super(…)` with arguments, or it does not compile.

Lesson: super and the order of construction

REMEMBER

When two interfaces give the same default method, the class must override it and may pick a version with `Wifi.super.connect()`. Python resolves the same clash with its method resolution order, and JavaScript combines behaviour with mixins.

Lesson: Interfaces

IN THE EXAM

Testers meet encapsulation in the Page Object Model: WebElements are private fields of a page class, and tests call public methods such as `login(user, pass)` instead of touching locators directly.

Lesson: Encapsulation with getters and setters

1.5 Strings, Arrays, Collections and Exceptions

Automation code spends most of its time on text, lists of values and things that go wrong. This chapter covers the parts of the Java library that handle them, using the session's programs: comparing strings, reading a price such as USD 59,900, counting unique values with a map, and keeping a test running when a line throws.

KEY IDEAS

- **Strings, the pool and equals** A String is an object holding a fixed sequence of characters. String literals are stored once in the String pool , so two variables with the same literal share one object. new String("java") always makes a separate object on the heap.
- **String methods**
- **Classic string programs** The session's string programs come up in almost every interview. Each one is a loop over characters or words plus one String method.
- **Arrays** An array holds a fixed number of values of one type in consecutive slots. new int[5] fills every slot with the default 0, and the size never changes afterwards. Reading animals[2] is one step, O(1), whatever the size.
- **Wrapper classes** Collections hold objects only, so List<Integer> stores wrappers. Java converts automatically: autoboxing wraps an int into an Integer, unboxing unwraps it. Testers use the parse methods constantly, because everything read from a web page is text.
- **The Collection framework**
- **Exception handling** An exception is an object describing a problem that stops normal flow, such as dividing by zero. Without handling, the program ends and prints a stack trace (the session's ExceptionSession7). With try and catch it recovers and carries on.
- **Checked and unchecked, throw and throws**

FORMULAS

Classic string programs Exam favourite

$$\frac{n^2}{2}$$

Lesson: Classic string programs

TABLES

Table: method · example · result

Method	Example	Result
`length()`	`"java is difficult".length()`	17
`charAt(i)`	`"automation".charAt(4)`	'm' (index starts at 0)
`substring(b, e)`	`"automation".substring(3, 7)`	"omat": from b up to, not including, e
`indexOf / lastIndexOf`	`"test".indexOf('a')`	–1 when not found
`contains / startsWith / endsWith`	`s.startsWith("Java")`	case sensitive true or false
`replace / replaceAll`	`price.replaceAll("[^0-9]", "")`	replaceAll takes a regular expression
`isEmpty / isBlank`	`" ".isEmpty(), " ".isBlank()`	false, true (isBlank ignores spaces)
`trim(), split(regex)`	`"a b".split(" ")`	["a", "b"]
`String.format`	`String.format("%s is %d", "Tom", 30)`	"Tom is 30"

Lesson: String methods

Table: program · idea

Program	Idea
Reverse	Loop from the last index down, or `new StringBuilder(s).reverse()`
Palindrome	Equal to its own reverse, after lower casing
camelCase	Split on `[^a-zA-Z0-9]`, lower case the first word, capitalise the rest
Remove duplicates	Keep the first appearance of each character: "automation" becomes "automin"

Lesson: Classic string programs

Table: primitive · wrapper · parse from text

Primitive	Wrapper	Parse from text
int	Integer	`Integer.parseInt("59900")`
double	Double	`Double.parseDouble("60000.05")`
boolean	Boolean	`Boolean.parseBoolean("false")`
char, long, float, byte, short	Character, Long, Float, Byte, Short	`Long.parseLong` and so on

Lesson: Wrapper classes

Table: type · duplicates · order · typical use in tests

Type	Duplicates	Order	Typical use in tests
ArrayList	Yes	Insertion, with index access	All product names on a page
LinkedList	Yes	Insertion, fast at both ends	A queue of steps
HashSet	No	None you can rely on	Checking for duplicate IDs
LinkedHashSet	No	Insertion	Unique values in the order seen
TreeSet	No	Sorted	Verifying a sorted dropdown
HashMap	Keys no, values yes	None you can rely on	Counting, config key to value

Lesson: The Collection framework

Table: block · rule

Block	Rule
try	Holds the risky lines. Must be followed by at least one catch or a finally
catch	Several are allowed. List the specific type before the general one: `catch (Exception)` above `catch (ArithmeticException)` is a compilation error, since the second could never run
finally	Always runs, with or without an exception, even after `return`. Use it to close drivers, files and connections

Lesson: Exception handling

Table: keyword · where · meaning

Keyword	Where	Meaning
`throw`	Inside a method body	Raises one exception object now: `throw new InvalidAgeException("...")`
`throws`	In the method signature	Warns callers which checked exceptions may come out

Lesson: Checked and unchecked, throw and throws

TIPS AND TRAPS

REMEMBER

Strings are **immutable**. Methods such as `concat`, `replace` and `toUpperCase` return a new String and leave the original unchanged. To build text in a loop, use `StringBuilder`, which changes one buffer instead of creating a new String on every pass.

Lesson: Strings, the pool and equals

REMEMBER

Joining Strings with += inside a loop, as the session's ReverseAString does, creates a new String every pass, so n characters cost about

$$\frac{n^2}{2}$$

character copies. StringBuilder keeps it to about n.

Lesson: Classic string programs

COMMON MISTAKE

Valid indexes run from 0 to `length - 1`. `nums[5]` on a 5 element array throws `ArrayIndexOutOfBoundsException` at run time. Note that `length` is a field for arrays and `length()` a method for Strings.

Lesson: Arrays

COMMON MISTAKE

`Integer` objects from −128 to 127 are cached, so == works by luck for small values and fails at 128. Always compare wrappers with `equals()`.

Lesson: Wrapper classes

REMEMBER

Use generics such as `List<String>`. The session's raw `List` accepts any type, so a wrong value is only discovered when the program crashes, not when it compiles.

Lesson: The Collection framework

IN THE EXAM

The session's ExceptionSession6 catches an exception and throws `new RuntimeException("Denominator is zero")`. That wraps the problem in a clearer message, a pattern frameworks use to turn low level errors into readable test failures.

Lesson: Checked and unchecked, throw and throws

1.6 Reading Test Data Files

Hard coding a username inside a test means editing code every time the data changes. The session's last module moves data into files under src/test/resources. This chapter reads each kind, text, properties, CSV, JSON and Excel, with the same file contents in Java, Python and JavaScript, and ends by feeding rows into one data driven test.

KEY IDEAS

- **Why test data lives in files** A data driven test keeps its inputs and expected results outside the code. Testers can add a row to a sheet without touching Java, the same test runs once per row, and secrets such as passwords stay out of the source.
- **Text files**
- **Properties files for configuration** Every value comes back as a String , so numbers need Integer.parseInt . Lines starting with # or ! are comments.
- **CSV rows**
- **JSON with Jackson and JsonPath** The session reads this file with JsonPath : JsonPath.parse(file).read("$.firstName") . $ is the root, .key goes into an object and [n] into an array.
- **Excel with Apache POI and Fillo** The session uses Fillo , which lets you query a sheet like a database table: select username, password from sheet1 where status='Pass' . Fillo is built on Apache POI , the standard Java library for Office files.
- **From rows to a data driven test** TestNG's @DataProvider returns Object[][] : one inner array per run, holding that run's arguments. Read any file into that shape and a single test method covers every row.

TABLES

Table: format · best for · java reader

Format	Best for	Java reader
.txt	Lists, logs, simple lines	`Files.readAllLines`, `BufferedReader`
.properties	Configuration: URL, browser, timeout	`java.util.Properties`
.csv	Rows of test data, easy to diff in Git	`split(",", -1)` or a CSV library
.json	Nested data, API payloads and responses	Jackson, JsonPath
.xlsx / .xls	Data that business users edit	Apache POI, Fillo

Lesson: Why test data lives in files

Table: jsonpath · jackson · value

JsonPath	Jackson	Value
`$.firstName`	`root.get("firstName").asText()`	John
`$.address.city`	`root.path("address").path("city").asText()`	New York
`$.phoneNumbers[1].number`	`root.at("/phoneNumbers/1/number")`	646 555-4567
`$.phoneNumbers[*].type`	loop over `root.get("phoneNumbers")`	[home, office, mobile]

Lesson: JSON with Jackson and JsonPath

Table: username · password · status · attempts

username	password	status	attempts
user1	Psw1	Pass	1
user2	Psw2	Fail	3
user3	Psw3	Pass	2
user4	Psw4	Fail	1
user5	Psw5	Pass	4

Lesson: Excel with Apache POI and Fillo

TIPS AND TRAPS

REMEMBER — The session builds every path from `System.getProperty("user.dir")`, the folder the program was started from, which is the project root in Maven and IntelliJ. Then `"/src/test/resources/test.txt"` works on every machine, unlike `C:\Users\me\...`.
Lesson: Why test data lives in files

REMEMBER — `try (BufferedReader br = …)` is try with resources: Java closes the file when the block ends, even if an exception is thrown. Python's `with open(…)` does the same.
Lesson: Text files

COMMON MISTAKE — Java's `split(",")` drops **trailing** empty fields: `"user5,Psw5,".split(",")` has 2 parts, not 3. Pass a limit of −1, `split(",", -1)`, to keep them. Values that contain commas inside quotes need a real CSV parser such as OpenCSV, or Python's `csv` module.
Lesson: CSV rows

REMEMBER — `get("email")` returns `null` for a missing key, so chaining `.asText()` after it throws a NullPointerException. `path("email")` returns a "missing node" whose text is empty, which is safer for deep chains.
Lesson: JSON with Jackson and JsonPath

REMEMBER — `WorkbookFactory.create` opens both the older `.xls` format (POI's HSSF classes) and `.xlsx` (XSSF classes, which need the `poi-ooxml` dependency). The Java example reads the `.xls` copy of the sheet, the Python and JavaScript versions read the `.xlsx` copy, and the code is otherwise the same for both formats. `DataFormatter` returns every cell as the text Excel shows, so a number cell such as 1 does not come back as 1.0.
Lesson: Excel with Apache POI and Fillo

IN THE EXAM — Keep an **expected** column next to the inputs, as login.csv does. A data row without its expected result can only tell you that the code ran, not that it did the right thing.
Lesson: From rows to a data driven test

1.7 Practice Program Bank

The session closes with a list of 66 Java programs that come up again and again in tester interviews. Many were solved in earlier chapters. This page solves the rest, groups them so that one program covers several related tasks, and gives you a checklist that links every item to its solution.

KEY IDEAS

- **The whole list in one place** Each row below is one program from the session's list. The link opens the chapter and block where it is solved. Programs marked "this page" are solved in the sections that follow, grouped so that one program handles several related tasks.
- **Arithmetic, swaps and characters** 15 / 2 is 7 in Java because both operands are int. Cast one of them first, (double) a / b , to get 7.5. Casting the result instead, (double) (a / b) , gives 7.0, because the fraction was already lost.
- **Number programs** A factor of n divides it with no remainder. Counting digits uses a do while loop so that 0 counts as one digit. The closed path program counts the loops in each digit: 0, 4, 6 and 9 have one, 8 has two, so 6784 → 1 + 0 + 2 + 1 = 4 .
- **Reverse, palindrome, Armstrong and neon** Reversing uses n % 10 to take the last digit and n / 10 to drop it. Java's % keeps the sign, so −123 becomes −321 with no special case, and 120 becomes 21 because a leading zero has no value.
- **String programs** Two items need a careful reading. "aabbbccccddddda" must print a:2 b:3 c:4 d:4 a:1 , so it counts runs of the same letter, not totals, which is why the final a stands alone.

- **Repeated characters and words** "First repeating character" has two common meanings, and interviewers accept either if you say which one you chose. In "data driven tests" the first letter met for a second time while scanning is a , at index 3.
- **Array programs**
- **Running totals and numbers beyond long** The running total item starts at 5 and adds each value of [−4, 1, 3, 5] : 1, 2, 5, 10 , then adds those results: 1 + 2 + 5 + 10 = 18 .

FORMULAS

Number programs Exam favourite

$$6784 \rightarrow 1 + 0 + 2 + 1 = 4$$

Lesson: Number programs

Armstrong Exam favourite

$$1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$$

Lesson: Reverse, palindrome, Armstrong and neon

Neon

$$9^2 = 81$$

Lesson: Reverse, palindrome, Armstrong and neon

Neon

$$8 + 1 = 9$$

Lesson: Reverse, palindrome, Armstrong and neon

Running totals and numbers beyond long

Exam favourite

$$1, 2, 5, 10$$

Lesson: Running totals and numbers beyond long

Running totals and numbers beyond long

$$1 + 2 + 5 + 10 = 18$$

Lesson: Running totals and numbers beyond long

TABLES

Table: swap · code · catch

Swap	Code	Catch
Third variable	`t = x; x = y; y = t;`	Clear and always safe
Add and subtract	`x = x + y; y = x - y; x = x - y;`	`x + y` can overflow for large values
XOR	`x ^= y; y ^= x; x ^= y;`	No overflow, but fails if x and y are the same variable

Lesson: Arithmetic, swaps and characters

Table: task · idea

Task	Idea
Missing digits 0 to 9	Mark each seen digit in a `boolean[10]`, then list the unmarked ones
Zeros to the end	Copy the non-zeros forward in order. The remaining slots are already 0
Zeros to the front	The same idea, filling from the right end
Unique values	LinkedHashSet keeps the order seen, TreeSet sorts

Lesson: Array programs

TIPS AND TRAPS

REMEMBER

For every program, write it yourself first, then test the edges before comparing: 0, 1, negative values, the largest value of the type, an empty string, and a string with spaces only. Most of the bugs in these programs live at those edges, as the prime checks in the control flow chapter showed.

Lesson: The whole list in one place

REMEMBER

`Math.pow` always returns a double, so `Math.pow(3, 10)` prints 59049.0. For whole number powers, multiply in a loop with a long, or use the fast power method from the methods chapter.

Lesson: Number programs

REMEMBER `Set.add` returns false when the value is already there. That single call both checks and records, so the first repeated word is found in one pass.

Lesson: Repeated characters and words

COMMON MISTAKE In JavaScript, `[10, 9, 1].sort()` gives `[1, 10, 9]`, because the default sort compares text. Always pass `(a, b) => a - b` for numbers.

Lesson: Array programs

IN THE EXAM Python's int has no size limit, and JavaScript has BigInt with literals such as 12n. In Java, `Long.MAX_VALUE + 1` silently wraps to the most negative long, with no exception.

Lesson: Running totals and numbers beyond long

STEP 2 Java Basics

2.1 Language Fundamentals

Every Java program is built from the same small bricks: names, types, values and arrays. Learn the bricks well and the big buildings get easy.

KEY IDEAS

- **Identifiers** An identifier is any name you invent in a program. It can name a class, a method, a variable or a label.
- **Reserved words** Some words already have a fixed meaning in Java, so you cannot use them as names. Classic Java has 53 reserved words : 50 keywords plus 3 reserved literals.
- **The 8 primitive data types** Java is strongly typed . Every variable and every expression has a type, and the compiler checks every assignment. Apart from the 8 primitive types everything is an object.
- **Literals** A literal is a fixed value written straight into the code. In int x = 10; the literal is 10 .
- **Widening and narrowing** Java converts a smaller type into a bigger one automatically. This is called widening or implicit type casting, and it never loses information about the size of the number.
- **Arrays** An array is an indexed group of a fixed number of values of the same type. Its biggest strength is reading or writing any slot instantly by index. Its biggest weakness is that the size can never change after creation.
- **Types of variables**
- **Var-arg methods** Before Java 1.5, a method had a fixed number of parameters. Need to add 2, 3 or 10 numbers and you wrote several methods. Var-arg methods accept any number of arguments, including zero.
- **The main() method** The compiler does not care whether a class has a main() method. The JVM checks it at run time, and if it cannot find the exact form it stops with an error.
- **Command line arguments** Arguments typed after the class name are passed to main as a String[] . They let you change a program's behaviour without changing its code.
- **Java coding standards** Good names make code read like a story. Java has well known naming habits that every team expects.

FORMULAS

Spot the identifiers

```
class Test {                          // Test is an
                    identifier
    public static void main(String[] args) {    //
        main, String, args are identifiers
        int x = 10;                       // x is an
                    identifier
                          }
                      }
```

Lesson: Identifiers

var is still a legal variable name

```
          var var = "contextual keyword";
             System.out.println(var);
```

Lesson: Reserved words

The 8 primitive data types

Exam favourite

$$\text{range of an n-bit signed type} = -2^{n-1} \text{ to } 2^{n-1} - 1$$

Lesson: The 8 primitive data types

The 8 primitive data types

$$\text{byte: } n = 8 \rightarrow -2^7 \text{ to } 2^7 - 1 = -128 \text{ to } 127$$

Lesson: The 8 primitive data types

Ask Java for the limits

```
System.out.println(Byte.MIN_VALUE + " to " +
                Byte.MAX_VALUE);
System.out.println(Short.MIN_VALUE + " to " +
                Short.MAX_VALUE);
System.out.println(Integer.MIN_VALUE + " to " +
                Integer.MAX_VALUE);
      System.out.println(Long.MAX_VALUE);
System.out.println((int) Character.MAX_VALUE);
```

Lesson: The 8 primitive data types

Literals

Exam favourite

$$010_8 = 1 \times 8^1 + 0 \times 8^0 = 8$$

Lesson: Literals

Literals

$$0x10_{16} = 1 \times 16^1 + 0 \times 16^0 = 16$$

Lesson: Literals

Same digits, different bases

```
    int a = 10, b = 010, c = 0x10, d = 0b10;
System.out.println(a + " " + b + " " + c + " " +
                d);
   System.out.println(0xFace + " " + 0XBeef);
```

Lesson: Literals

Widening and narrowing

Exam favourite

$$\text{byte} \rightarrow \text{short} \rightarrow \text{int} \rightarrow \text{long} \rightarrow \text{float} \rightarrow \text{double}$$

Lesson: Widening and narrowing

Widening and narrowing

$$\text{char} \rightarrow \text{int} \rightarrow \text{long} \rightarrow \text{float} \rightarrow \text{double}$$

Lesson: Widening and narrowing

Widening and narrowing

$$130 = 1000\ 0010_2 \rightarrow \text{as a signed byte} = -128 + 2 = -126$$

Lesson: Widening and narrowing

Narrowing with a cast

```
                int big = 130;
byte b = (byte) big;          // keeps only the
                lowest 8 bits
            System.out.println(b);
               double d = 9.99;
 int whole = (int) d;           // the decimal
        part is cut off, not rounded
            System.out.println(whole);
```

Lesson: Widening and narrowing

Java snippet

```
int[] a;     // recommended: the type int[] is
        clearly separate from the name
                    int a[];
                    int []a;
```

Lesson: Arrays

Static code cannot see instance variables directly

```
                    class Test {
                       int x = 10;
                    static int y = 20;
          public static void main(String[] args) {
                    Test t = new Test();
             System.out.println(t.x + " " + y);    // x
                needs an object, y does not
                                }
                              }
```

Lesson: Types of variables

Var-arg must be last

```
                    class Main {
          static void m(int... x, String s) { }
        public static void main(String[] args) { }
                              }
```

Lesson: Var-arg methods

A dressed up but valid main

```
                    class Main {
       final static synchronized strictfp public
          void main(final String... hi) {
             System.out.println("Still runs");
                              }
                            }
```

Lesson: The main() method

run as: java Main A B

```
                System.out.println(args[0]);
                System.out.println(args[1]);
                System.out.println(args[2]);
```

Lesson: Command line arguments

TABLES

Table: name · valid · why

Name	Valid	Why
java2share	Yes	Digits are fine after the first character
4shares	No	Starts with a digit
all@hands	No	@ is not allowed
total_number_of_students	Yes	Underscores are allowed
_$_	Yes	Only allowed characters, and it is not the lone _
total#	No	# is not allowed
int	No	Reserved word
Integer	Yes	A class name, legal but a bad idea

Lesson: Identifiers

Table: type · size · range · default

Type	Size	Range	Default	Wrapper class
`byte`	1 byte	−128 to 127	0	Byte
`short`	2 bytes	−32,768 to 32,767	0	Short
`int`	4 bytes	−2,147,483,648 to 2,147,483,647	0	Integer
`long`	8 bytes	-2^{63} to $2^{63} - 1$	0	Long
`float`	4 bytes	about $\pm 3.4 \times 10^{38}$, 6 to 7 digits of precision	0.0	Float
`double`	8 bytes	about $\pm 1.7 \times 10^{308}$, 15 to 16 digits of precision	0.0	Double
`char`	2 bytes	0 to 65,535 (Unicode)	'\u0000'	Character
`boolean`	not fixed (JVM decides)	`true` or `false`	false	Boolean

Lesson: The 8 primitive data types

Table: form · prefix · digits allowed · example

Form	Prefix	Digits allowed	Example	Value
Decimal	none	0 to 9	`10`	10
Octal	`0`	0 to 7	`010`	8
Hexadecimal	`0x` or `0X`	0 to 9, a to f (any case)	`0x10`	16
Binary (Java 7)	`0b` or `0B`	0 and 1	`0b10`	2

Lesson: Literals

Table: · instance · static · local

	Instance	Static	Local
Declared	inside the class, outside methods	same, with `static`	inside a method, block or constructor
Copies	one per object	one per class, shared by all objects	one per method call
Created	when the object is created	when the class is loaded	when the block starts
Destroyed	when the object is garbage collected	when the class is unloaded	when the block ends
Stored	inside the object on the heap	with the class data	on the thread's stack
Default value	yes	yes	no, you must set it
Modifiers allowed	all field modifiers	all field modifiers	only `final`

Lesson: Types of variables

Table: word · why it is there

Word	Why it is there
`public`	so the JVM can call it from outside the class
`static`	so the JVM can call it without creating an object first
`void`	main returns nothing to the JVM
`main`	the name the JVM is programmed to look for
`String[] args`	the command line arguments

Lesson: The main() method

Table: what · style · usually · examples

What	Style	Usually	Examples
Class	PascalCase	a noun	`Student`, `StringBuilder`
Interface	PascalCase	an adjective	`Runnable`, `Serializable`, `Comparable`
Method	camelCase	a verb or verb plus noun	`print()`, `getName()`, `setSalary()`
Variable	camelCase	a noun	`name`, `rollNo`, `mobileNumber`
Constant	UPPER_CASE with _	a noun	`MAX_VALUE`, `PI`
Package	all lowercase, reversed domain		`com.brainybytes.util`

Lesson: Java coding standards

TIPS AND TRAPS

REMEMBER

Every reserved word is written in lowercase. `NULL`, `True` and `String` are not reserved words.

Lesson: Reserved words

IN THE EXAM

Words that sound like keywords but are not: `main`, `String`, `length`, `delete`, `include`, `sizeof`, `then`, `exit`. In Java the keyword is `extends`, never `extend`, and `implements`, never `implement`.

Lesson: Reserved words

COMMON MISTAKE

Money and float do not mix. `0.1 + 0.2` is not exactly `0.3` in binary floating point. Use `BigDecimal` for money.

Lesson: The 8 primitive data types

REMEMBER

There is no way to write a byte or short literal directly. When an int literal fits in the range, Java quietly lets you store it in a `byte` or `short`.

Lesson: Literals

IN THE EXAM

Floating point literals can only be written in decimal. `double d = 0x123.4;` is not allowed, but `double d = 0x123;` is fine because `0x123` is an int that gets widened.

Lesson: Literals

REMEMBER

An int **constant** may be stored in a byte, short or char without a cast if its value fits. A plain int **variable** may not, because the compiler cannot be sure what value it holds. Mark the variable `final` and it becomes a constant again.

Lesson: Widening and narrowing

IN THE EXAM

`int[] a, b;` makes both arrays one dimensional. `int[] a[], b;` makes `a` two dimensional and `b` one dimensional. A dimension written before a name only works for the first name, so `int[] []a, []b;` does not compile.

Lesson: Arrays

REMEMBER

When you run `java Test`, the JVM starts, creates the main thread, finds `Test.class`, loads it (static variables are born here), runs `main()`, then unloads the class and shuts down.

Lesson: Types of variables

IN THE EXAM

An array's elements always get default values, even when the array is a local variable. `int[] a = new int[3];` inside a method gives three zeros. Only the variable `a` itself needs to be initialized.

Lesson: Types of variables

REMEMBER

`main(String[] args)` can be written as `main(String... args)`. And `int[]... x` is the same as `int[][] x`.

Lesson: Var-arg methods

IN THE EXAM `public static int main(String[] args)` compiles fine, but the JVM refuses to start it, because the return type must be `void`. Since Java 7, a class with only a static block and no main also refuses to run.

Lesson: The main() method

2.2 Operators and Assignments

Operators are the verbs of Java. They add, compare, join and decide. A few of them have sneaky rules, and those rules are exactly what interviewers love to ask about.

KEY IDEAS

- **Increment and decrement** ++ adds 1 to a variable and -- takes 1 away. Each comes in two flavours.
- **Arithmetic operators** The arithmetic operators are + - * / % . The type of the result follows one simple rule.
- **String concatenation** The + operator is the only overloaded operator in Java. If at least one side is a String it joins text, otherwise it adds numbers. Java works from left to right , so the moment a String appears, every + after it joins text.
- **Relational operators** < <= > >= compare numbers and chars. They work on every primitive type except boolean , and never on objects.
- **Equality operators** == and != work on every primitive, including boolean , and also on object references.
- **The instanceof operator** instanceof checks whether an object is of a certain type, which includes its parent classes and interfaces.
- **Bitwise and boolean operators** The operators & | ^ work on whole numbers bit by bit, and on booleans as logic. ~ flips every bit of a whole number. ! flips a boolean.
- **Short-circuit operators** && and || are the lazy cousins of & and | . They skip the right side when the left side already decides the answer.
- **Type casting**
- **Assignment operators** Every compound assignment hides a cast back to the type on the left. b += 1 means b = (byte)(b + 1) .
- **The conditional operator** The only ternary operator in Java takes three parts: a condition, a value for true and a value for false. It is a compact if else that produces a value.
- **new, [ ] and the evaluation order** new creates objects and [] declares or indexes arrays. Java has no delete operator, because the garbage collector removes unused objects.

FORMULAS

See it happen

```
                int x = 10;
               int y = x++;
System.out.println("x = " + x + ", y = " + y);
                int a = 10;
               int b = ++a;
System.out.println("a = " + a + ", b = " + b);
```

Lesson: Increment and decrement

Arithmetic operators Exam favourite

result type = the biggest of (int, type of a, type of b)

Lesson: Arithmetic operators

Arithmetic operators

$a \% b = a - b \times (a / b)$, where a / b is cut towards zero

Lesson: Arithmetic operators

Arithmetic operators

$-17 \% 5 = -17 - 5 \times (-3) = -17 + 15 = -2$

Lesson: Arithmetic operators

chars turn into numbers

```
System.out.println('a' + 'b');
System.out.println('a' + 1);
System.out.println((char) ('a' + 1));
System.out.println(10 / 3 + " " + 10 / 3.0);
```

Lesson: Arithmetic operators

String concatenation

Exam favourite

$$a + b + c + d = ((10 + 20) + 30) + \text{"java"} = 60 + \text{"java"} = \text{"60java"}$$

Lesson: String concatenation

String concatenation

$$a + b + d + c = (30 + \text{"java"}) + 30 = \text{"30java"} + 30 = \text{"30java30"}$$

Lesson: String concatenation

Where the String appears changes everything

```
int a = 10, b = 20, c = 30;
String d = "java";
System.out.println(a + b + c + d);
System.out.println(d + a + b + c);
System.out.println(a + b + d + c);
System.out.println(d + (a + b) + c);
```

Lesson: String concatenation

Comparing numbers and chars

```
System.out.println(10 < 20);
System.out.println('a' < 'b');
System.out.println('a' > 10.5);
System.out.println(10.5 ⩾ 10);
```

Lesson: Relational operators

Primitives compare values

```
System.out.println(10 == 20);
System.out.println('a' == 97.0);
System.out.println(false == false);
```

Lesson: Equality operators

Checking the real type

```
Object o = new ArrayList<String>();
System.out.println(o instanceof ArrayList);
System.out.println(o instanceof List);
System.out.println(o instanceof Collection);
System.out.println(o instanceof Map);
System.out.println(null instanceof Object);
```

Lesson: The instanceof operator

Bitwise and boolean operators

Exam favourite

$$4 = 100_2,\ 5 = 101_2 \rightarrow 4\ \&\ 5 = 100_2 = 4,\ 4 \mid 5 = 101_2 = 5,\ 4 \wedge 5 = 001_2 = 1$$

Lesson: Bitwise and boolean operators

Bitwise and boolean operators

$$\sim x = -x - 1, \text{ so } \sim 4 = -5$$

Lesson: Bitwise and boolean operators

Bit by bit

```
System.out.println(4 & 5);
System.out.println(4 | 5);
System.out.println(4 ^ 5);
System.out.println(~4);
System.out.println(true ^ false);
System.out.println(!true);
```

Lesson: Bitwise and boolean operators

Count the evaluations

```
int x = 10, y = 15;
if (++x < 10 & ++y > 15) x++;
else y++;
System.out.println(x + " " + y);

int p = 10, q = 15;
if (++p < 10 && ++q > 15) p++;
else q++;
System.out.println(p + " " + q);
```

Lesson: Short-circuit operators

Type casting

Exam favourite

$$130 = 1000\ 0010_2 \rightarrow \text{as a byte: } -128 + 2 = -126$$

Lesson: Type casting

What gets lost

```
int x = 130;
byte b = (byte) x;
System.out.println(b);
double d = 130.456;
int i = (int) d;
byte b2 = (byte) d;
System.out.println(i + " " + b2);
System.out.println((int) -9.99);
```

Lesson: Type casting

Assignment operators

Exam favourite

$$d /= 2 \rightarrow d = 10$$

Lesson: Assignment operators

Assignment operators

$$c *= d \rightarrow c = 20 \times 10 = 200$$

Lesson: Assignment operators

Assignment operators

$$b -= c \rightarrow b = 20 - 200 = -180$$

Lesson: Assignment operators

Assignment operators

$$a += b \rightarrow a = 20 + (-180) = -160$$

Lesson: Assignment operators

Chained assignment

```
int a, b, c, d;
a = b = c = d = 20;
System.out.println(a + b + c + d);
```

Lesson: Assignment operators

Choose a value

```
int marks = 72;
String result = marks ⩾ 35 ? "Pass" : "Fail";
System.out.println(result);
int x = 5;
String size = x > 10 ? "big" : x > 3 ? "medium" :
"small";
System.out.println(size);
```

Lesson: The conditional operator

new, [ ] and the evaluation order

Exam favourite

$$1 + ((2 \times 3) / 4) \times 5 + 6 = 1 + (6 / 4) \times 5 + 6 = 1 + 1 \times 5 + 6 = 12$$

Lesson: new, [ ] and the evaluation order

Left to right, then precedence

```
class Main {
static int m(int i) {
System.out.print(i + " ");
return i;
}
public static void main(String[] args) {
int r = m(1) + m(2) * m(3) / m(4) * m(5)
+ m(6);
System.out.println("= " + r);
}
}
```

Lesson: new, [ ] and the evaluation order

TABLES

Table: expression · x before · x after · y

Expression	x before	x after	y
y = ++x;	10	11	11
y = x++;	10	11	10
y = --x;	10	9	9
y = x--;	10	9	10

Lesson: Increment and decrement

Table: operation · result type

Operation	Result type
byte + byte	int
byte + short	int
char + char	int
int + long	long
long + float	float
char + double	double

Lesson: Arithmetic operators

Table: operator · name · gives 1 (true) when

Operator	Name	Gives 1 (true) when
&	AND	both bits are 1
|	OR	at least one bit is 1
^	XOR	the bits are different
~	complement	flips every bit (integral types only)
!	NOT	flips a boolean (boolean only)

Lesson: Bitwise and boolean operators

Table: · & and | · && and ||

	& and |	&& and ||
Right side evaluated	always	only when needed
Speed	slower	faster
Works on	booleans and whole numbers	booleans only

Lesson: Short-circuit operators

Table: group · operators

Group	Operators
Unary	`[]` `x++` `x--`, then `++x` `--x` `~` `!`, then new and casts
Multiplicative	`*` `/` `%`
Additive	`+` `-`
Shift	`<<` `>>` `>>>`
Relational	`<` `<=` `>` `>=` `instanceof`
Equality	`==` `!=`
Bitwise	&, then ^, then |
Short-circuit	&&, then ||
Conditional	`?:`
Assignment	`=` `+=` `-=` *= and the rest

Lesson: new, [ ] and the evaluation order

TIPS AND TRAPS

COMMON MISTAKE Integer overflow is silent. `Integer.MAX_VALUE + 1` becomes `Integer.MIN_VALUE` without any error. Use `Math.addExact` or `long` when numbers can grow large.

Lesson: Arithmetic operators

IN THE EXAM Every comparison with `NaN` is false, even `NaN == NaN`. The only one that is true is `!=`. Use `Double.isNaN(x)` to test for it.

Lesson: Arithmetic operators

REMEMBER Parentheses run first, so `"Sum: " + (a + b)` prints `Sum: 30` while `"Sum: " + a + b` prints `Sum: 1020`.

Lesson: String concatenation

REMEMBER To compare two references with ==, their types must be related (parent and child, or the same type). Otherwise the compiler stops you with "incomparable types". And `r == null` is always allowed.

Lesson: Equality operators

REMEMBER Short-circuiting is how you guard against crashes: `if (s != null && s.length() > 0)` never calls `length()` on a null reference.

Lesson: Short-circuit operators

REMEMBER Nest it at most once. Beyond that a normal `if else` is kinder to the next reader.

Lesson: The conditional operator

REMEMBER Precedence decides how the expression is grouped. The **operands** are still evaluated strictly from left to right.

Lesson: new, [ ] and the evaluation order

2.3 Flow Control

Flow control decides which line runs next. Choose a road with if and switch, go round with loops, and jump out with break and continue.

KEY IDEAS

- **Three families of flow control** Java runs statements from top to bottom unless something changes the order. Flow control statements are those somethings, and they come in three families.
- **if else** In the last example b = false assigns false and then gives false, so the else part runs. Write b == false or simply !b .
- **switch** When one value decides between many paths, switch is cleaner than a long chain of if else if .
- **while loop** Use while when you do not know in advance how many times the body will run, for example reading lines until a file ends.
- **do while loop** A do while loop checks its condition at the end, so the body always runs at least once . A menu that shows itself before asking "again?" is the classic use.
- **for loop** The most used loop. Use it when you know how many times to repeat.
- **for-each loop** Java 5 added the for-each loop, the easiest way to visit every element of an array or a collection.
- **break** break jumps out of the nearest switch or loop immediately. It is also allowed inside a labeled block. Anywhere else it does not compile.
- **continue** continue skips the rest of the current pass and moves on to the next one. It is allowed only inside loops.
- **Labeled break and continue** In nested loops, a plain break or continue only affects the inner loop. Put a label on the outer loop to target it instead.

FORMULAS

Java snippet

```
if (condition) {
// runs when condition is true
} else {
// runs when condition is false (the else part is optional)
}
```

Lesson: if else

Classic switch

```
int day = 3;
switch (day) {
case 1: System.out.println("Monday"); break;
case 2: System.out.println("Tuesday"); break;
case 3: System.out.println("Wednesday"); break;
default: System.out.println("Some other day");
}
```

Lesson: switch

Halving until we reach 1

```
int n = 100, steps = 0;
while (n > 1) {
n = n / 2;
steps++;
}
System.out.println("steps = " + steps);
```

Lesson: while loop

Runs once even though the condition is false

```
int x = 10;
do {
System.out.println("x = " + x);
x++;
} while (x < 5);
```

Lesson: do while loop

Java snippet

```
for (initialization ; condition ; update) {
body
}
```

Lesson: for loop

Arrays and collections

```
int[] marks = {72, 88, 95};
for (int m : marks) System.out.print(m + " ");
System.out.println();
List<String> names = List.of("Asha", "Ravi", "Meera");
for (String n : names)
System.out.print(n.charAt(0));
System.out.println();
int[][] grid = {{1, 2}, {3, 4, 5}};
for (int[] row : grid)
for (int v : row) System.out.print(v);
```

Lesson: for-each loop

Stop at the first multiple of 7

```
for (int i = 1; i <= 20; i++) {
if (i % 7 == 0) {
System.out.println("found " + i);
break;
}
}
```

Lesson: break

Print only odd numbers

```
for (int i = 0; i < 10; i++) {
if (i % 2 == 0) continue;
System.out.print(i + " ");
}
```

Lesson: continue

break with a label leaves both loops

```
outer:
for (int i = 0; i < 3; i++) {
for (int j = 0; j < 3; j++) {
if (i == j) break outer;
System.out.println(i + "..." + j);
}
}
System.out.println("after the loops");
```

Lesson: Labeled break and continue

TABLES

Table: since · allowed types

Since	Allowed types
Java 1.0	`byte`, `short`, `char`, `int`
Java 5	their wrappers `Byte Short Character Integer`, and `enum`
Java 7	`String`
Java 21	any object, using pattern matching

Lesson: switch

Table: inside the inner loop · pairs printed for i, j in 0 to 2

Inside the inner loop	Pairs printed for i, j in 0 to 2
`break`	1…0, 2…0, 2…1
`break outer`	nothing
`continue`	0…1, 0…2, 1…0, 1…2, 2…0, 2…1
`continue outer`	1…0, 2…0, 2…1

Lesson: Labeled break and continue

TIPS AND TRAPS

REMEMBER — The condition must be a `boolean`. Java never treats 0 as false or 1 as true, unlike C.
Lesson: if else

IN THE EXAM — An `else` always belongs to the **nearest** unmatched `if`, whatever the indentation says. This is the "dangling else".
Lesson: if else

REMEMBER — `default` can sit anywhere, even at the top. It still runs only when no case matches, and fall-through still applies from wherever it is.
Lesson: switch

IN THE EXAM — If `a` and `b` were `final`, then `a > b` would become the constant `false` and the loop body would be an unreachable statement.
Lesson: while loop

REMEMBER — `for (;;)` is an endless loop, the same as `while (true)`.
Lesson: for loop

IN THE EXAM — In a `do while` loop, `continue` jumps to the condition check at the bottom, not to the top of the body. Trace the next example by hand before you look at the output.
Lesson: continue

STEP 3 Object Oriented Java

3.1 Declarations and Access Modifiers

Modifiers are the small words in front of a class or member that decide who may see it, who may change it and how it behaves. Get them right and your code explains itself.

KEY IDEAS

- **Java source file structure**

- **The import statement** Without an import you must write the full name of a class every time: java.util.ArrayList<String> list = new java.util.ArrayList<>(); . An import lets you use the short name.
- **Static import** Java 5 added static import , which lets you use static members without writing the class name.
- **Packages** A package groups related classes and interfaces, the way folders group files. It solves name clashes, improves modularity and adds a layer of access control.
- **Class level modifiers** A top level class may use only these modifiers: public , default (no word), final , abstract and strictfp . Inner classes may also be private , protected and static .
- **final** JVM does not give final fields a default value. You must assign them yourself, exactly once, at one of these places:
- **abstract** An abstract method says what a child must do, without saying how. It has no body and ends with a semicolon. An abstract class is a class that may contain such methods and cannot be instantiated.
- **public, protected, default and private** These four control who can see a field, method or constructor. From most restricted to most open:
- **static, native, synchronized, transient, volatile, strictfp** A static member belongs to the class, not to any object. It can be used through the class name without creating an object. Static code cannot touch instance members directly, because there may be no object at all.
- **Interfaces** An interface is a contract: a list of services a class promises to provide. The person using it only needs the contract, not the implementation. JDBC is a classic example: Java defines interfaces like Connection , and each database vendor writes the classes.
- **Naming conflicts in interfaces** If two interfaces declare the same method with the same signature and return type, one implementation in the class satisfies both. If they have the same signature but different return types, no class can implement both.
- **Marker interfaces and adapter classes** A marker interface has no methods at all. Implementing it simply tags your class with an ability that the JVM or a library checks for.
- **Abstract class or interface**

FORMULAS

Three classes, one file, three .class files

```
class A {
public static void main(String[] args) {
  System.out.println("A's main"); }
}
class B {
public static void main(String[] args) {
  System.out.println("B's main"); }
}
class C { }
```

Lesson: Java source file structure

Date lives in two packages

```
import java.util.*;
import java.sql.*;
class Main {
public static void main(String[] args) {
  Date d = new Date();
  }
}
```

Lesson: The import statement

Math and System without their class names

```
import static java.lang.Math.*;
import static java.lang.System.out;
class Main {
public static void main(String[] args) {
  out.println(sqrt(16) + " " + max(10, 20)
  + " " + PI);
  }
}
```

Lesson: Static import

A class inside a package

```
package com.brainybytes.demo;
public class Main {
public static void main(String[] args) {
  System.out.println("Hello from " +
    Main.class.getName());
  }
}
```

Lesson: Packages

A top level class cannot be private

```
private class Main {
public static void main(String[] args) { }
}
```

Lesson: Class level modifiers

Extending a final class

```
final class P { }
class C extends P {
public static void main(String[] args) { }
}
```

Lesson: final

Cannot create an abstract class

```
abstract class Shape { }
class Main {
public static void main(String[] args) {
Shape s = new Shape(); }
}
```

Lesson: abstract

public, protected, default and private

Exam favourite

private < default < protected < public

Lesson: public, protected, default and private

Instance field from a static method

```
class Main {
int x = 10;
public static void main(String[] args) {
System.out.println(x);
}
}
```

Lesson: static, native, synchronized, transient, volatile, strictfp

Implementing without public

```
interface Greeter { void greet(); }
class Main implements Greeter {
void greet() { System.out.println("hi"); }
public static void main(String[] args) { }
}
```

Lesson: Interfaces

One method satisfies two interfaces

```
interface Left { void m(); }
interface Right { void m(); }
class Main implements Left, Right {
public void m() { System.out.println("one m
for both"); }
public static void main(String[] args) { new
Main().m(); }
}
```

Lesson: Naming conflicts in interfaces

clone without Cloneable

```
class Sheep {
public static void main(String[] args) throws
Exception {
Sheep dolly = new Sheep();
Object copy = dolly.clone();
}
}
```

Lesson: Marker interfaces and adapter classes

TABLES

Table: class modifier · meaning

Class modifier	Meaning
`public`	usable from any package
default	usable only inside its own package (package-private)
`final`	nobody can extend it. `String` and every wrapper class are final
`abstract`	nobody can create objects of it directly. It exists to be extended
`strictfp`	historic: forced strict IEEE floating point (see below)

Lesson: Class level modifiers

Table: final instance variable · final static variable

final instance variable	final static variable
at the declaration	at the declaration
inside an instance block `{ }`	inside a static block `static { }`
inside every constructor	

Lesson: final

Table: accessed from · private · default · protected

Accessed from	private	default	protected	public
the same class	✓	✓	✓	✓
another class, same package	×	✓	✓	✓
a child class, other package	×	×	✓ (through the child reference only)	✓
a non-child class, other package	×	×	×	✓

Lesson: public, protected, default and private

Table: can i use · from an instance method · from a static method

Can I use	from an instance method	from a static method
an instance member	yes	no (needs an object)
a static member	yes	yes

Lesson: static, native, synchronized, transient, volatile, strictfp

Table: modifier · class · method · variable

Modifier	Class	Method	Variable	Block
static	inner only	✓	✓	✓
native	×	✓	×	×
synchronized	×	✓	×	✓
transient	×	×	✓	×
volatile	×	×	✓	×
strictfp	✓	✓	×	×

Lesson: static, native, synchronized, transient, volatile, strictfp

Table: question · interface · abstract class

Question	Interface	Abstract class
Use it when you know	only the requirements (what)	part of the implementation (how)
Methods	abstract, plus default, static and private since Java 8 and 9	abstract and concrete
Fields	always `public static final` constants	any kind, including instance fields
Constructors	not allowed	allowed, run when a child object is created
Instance or static blocks	not allowed	allowed
Multiple inheritance	a class can implement many	a class can extend only one
Method modifiers	public (or private for helpers)	any access level

Lesson: Abstract class or interface

TIPS AND TRAPS

REMEMBER Keep one class per file and name the file after it. It makes code much easier to find, and every team expects it.

Lesson: Java source file structure

REMEMBER When the same static name is available from several places, Java picks in this order: a member of the current class first, then an explicit static import, then a wildcard static import.

Lesson: Static import

COMMON MISTAKE Static imports save typing but can hide where a name comes from. Use them sparingly, for things like `Math` functions or test assertions.

Lesson: Static import

REMEMBER The order of a source file is fixed: at most one `package`, then any number of `import` lines, then the classes, interfaces and enums. An empty file is also a valid Java file.

Lesson: Packages

REMEMBER A `final` method parameter cannot be reassigned inside the method. Local variables that are final behave like constants once set.

Lesson: final

IN THE EXAM `abstract` never talks about implementation, so it clashes with modifiers that do: `final`, `static`, `private`, `synchronized`, `native` and `strictfp` (in older Java). `abstract final` is the most common trap: one says "must be overridden", the other says "can never be overridden".

Lesson: abstract

REMEMBER The class itself must be visible first. A public method inside a default class is still unreachable from another package.

Lesson: public, protected, default and private

REMEMBER Common practice: fields `private`, the methods others need `public`, and `protected` only for hooks meant for subclasses.

Lesson: public, protected, default and private

IN THE EXAM "An abstract class cannot be instantiated, so why does it have a constructor?" Because creating a child object runs the parent constructor too, which initializes the fields the abstract class defines.

Lesson: Abstract class or interface

3.2 OOP Concepts

Object oriented programming is about building software from objects that keep their own data, promise clear services and can stand in for each other. This chapter is the heart of Java.

KEY IDEAS

- **Data hiding** Data hiding means outsiders cannot touch an object's data directly. They have to go through methods, and those methods can check every request. In Java you hide data by making fields private .
- **Abstraction** Abstraction means showing what something does while hiding how it does it. An ATM shows "withdraw" and "check balance", and hides the network calls and database updates behind the screen.
- **Encapsulation** Encapsulation wraps data and the code that works on it into one unit, a class. Every Java class is encapsulated. When it also hides its data (data hiding) and offers services instead (abstraction), we get the full benefit.
- **IS-A relationship (inheritance)** The IS-A relationship is inheritance, written with extends . A Dog IS-A Animal , so it gets everything an Animal has. The big win is code reuse .
- **HAS-A relationship** HAS-A means one object holds a reference to another. A Car HAS-A Engine . There is no special keyword, you just declare a field. It also reuses code, without the tight bond of inheritance.
- **Method signature** A method's signature is its name plus the types of its parameters, in order. The return type is not part of it. The compiler uses signatures to find the right method.
- **Overloading** Methods are overloaded when they share a name but have different parameter lists. The compiler picks one by looking at the argument types at compile time, so overloading is called compile-time or static polymorphism.
- **Overriding** A child overrides a parent method by declaring the same signature with its own body. Which body runs is decided at run time , by the actual object. That is why overriding is called runtime polymorphism, dynamic method dispatch or late binding.

- **Overloading versus overriding**
- **Polymorphism** Polymorphism means "many forms": one name, different behaviour depending on the situation. Java has two kinds.
- **Static control flow** When a class is loaded, Java runs its static parts in three steps:
- **Instance control flow** Every time you create an object, Java runs these steps:
- **Constructors** Creating an object is not enough. It also needs its starting values. A constructor runs automatically at creation to do exactly that.
- **Singleton classes** A singleton class allows only one object. Runtime is one: Runtime.getRuntime() always returns the same object. Singletons save memory when every user can share one object, such as a configuration or a connection pool.
- **Coupling and cohesion**
- **Object type casting** Casting an object reference changes the type of the reference , never the object itself.

FORMULAS

Direct access is refused

```
class Account { private double balance; }
class Main {
  public static void main(String[] args) {
    new Account().balance = 1_000_000;
  }
}
```

Lesson: Data hiding

Encapsulation

Exam favourite

Encapsulation = Data hiding + Abstraction

Lesson: Encapsulation

Java snippet

```
class A { private int x = 10; }          //
    tightly encapsulated
class B extends A { int y = 20; }        // not:
    y is not private
class C extends A { private int z = 30; } //
    tightly encapsulated
```

Lesson: Encapsulation

A parent reference does not know child methods

```
class Animal { }
class Dog extends Animal { void bark() { } }
class Main {
  public static void main(String[] args) {
    Animal a = new Dog();
    a.bark();
  }
}
```

Lesson: IS-A relationship (inheritance)

A car that has an engine

```
class Engine { String start() { return "vroom"; }
}
class Car {
  private final Engine engine = new Engine();
    // HAS-A
  void drive() { System.out.println("Engine
    says " + engine.start()); }
}
class Main {
  public static void main(String[] args) { new
    Car().drive(); }
}
```

Lesson: HAS-A relationship

Same signature, different return type

```
class Main {
  public void m1(int i) { }
  public int m1(int x) { return 10; }
}
```

Lesson: Method signature

One name, three methods

```
class Main {
  static void print(int i)    {
    System.out.println("int: " + i); }
  static void print(double d) {
    System.out.println("double: " + d); }
  static void print(String s) {
    System.out.println("String: " + s); }
  public static void main(String[] args) {
    print(10); print(10.5); print("ten");
  }
}
```

Lesson: Overloading

Covariant return type

```
class Parent { Object get() { return "parent"; }
}
class Child extends Parent {
  @Override String get() { return "child"; }
}
class Main {
  public static void main(String[] args) {
    System.out.println(new
      Child().get().length());
  }
}
```

Lesson: Overriding

Direct read during RIWO

```
class Main {
  static { System.out.println(x); }
  static int x = 10;
}
```

Lesson: Static control flow

void makes it a method, not a constructor

```
class Main {
  void Main() { System.out.println("method"); }
  public static void main(String[] args) {
    Main m = new Main();          // default
      constructor, prints nothing
      m.Main();                   // the
      method runs only when called
  }
}
```

Lesson: Constructors

Coupling and cohesion

Exam favourite

Good design = loose coupling + high cohesion

Lesson: Coupling and cohesion

Object type casting

Exam favourite

A b = (C) d;

Lesson: Object type casting

All three checks pass

```
Object o = new String("cast me");
String s = (String) o;
System.out.println(s.toUpperCase());
```

Lesson: Object type casting

TABLES

Table: statement · allowed

Statement	Allowed
Parent methods called on a child object	yes
Child methods called on a parent object	no
Parent reference holding a child object	yes
Child methods through a parent reference	no, the compiler only knows the parent type
Child reference holding a parent object	no

Lesson: IS-A relationship (inheritance)

Table: property · overloading · overriding

Property	Overloading	Overriding
Method names	same	same
Parameter types	must differ	must match
Return type	free	same or covariant
private, static, final methods	can be overloaded	cannot be overridden
Access modifiers	free	cannot become narrower
throws clause	free	no broader checked exceptions
Decided by	compiler, using reference types	JVM, using the runtime object
Also called	compile-time, static, early binding	runtime, dynamic, late binding

Lesson: Overloading versus overriding

Table: check · when · rule · if broken

Check	When	Rule	If broken
1	compile time	the type of `d` and `C` must be related (one is a parent of the other)	compile error: incompatible types
2	compile time	`C` must be the same as `A` or a child of `A`	compile error: incompatible types
3	run time	the real object in `d` must be a `C` or a child of `C`	ClassCastException

Lesson: Object type casting

TIPS AND TRAPS

REMEMBER — The main gain is **security**: nobody can set a negative balance or skip the checks.

Lesson: Data hiding

IN THE EXAM — If the parent class is not tightly encapsulated, no child can be either, because the child inherits the non-private field.

Lesson: Encapsulation

REMEMBER — "Favour HAS-A over IS-A" is common advice. Use inheritance only when the child truly is a kind of the parent.

Lesson: HAS-A relationship

IN THE EXAM — Overload resolution looks at the **reference** type, never the runtime object. With `m(Animal)` and `m(Monkey)`, the call `m(a)` where `Animal a = new Monkey();` picks `m(Animal)`.

Lesson: Overloading

REMEMBER — Write `@Override` above every overriding method. If you misspell the name, the compiler tells you instead of quietly creating a new method.

Lesson: Overriding

REMEMBER — During RIWO you may read a variable through a method (indirectly) but not by name directly. A direct read before the declaration is an "illegal forward reference".

Lesson: Static control flow

REMEMBER — Use a static block for one-time setup when the class loads, such as loading a native library or filling a lookup table.

Lesson: Static control flow

IN THE EXAM — Constructors and instance blocks both run at object creation. The instance blocks run first, then the constructor body.

Lesson: Constructors

REMEMBER

The same idea makes "doubleton" classes (two objects) and pools. And a `private` constructor alone also stops a class from being extended, because a child must call it.

Lesson: Singleton classes

REMEMBER

Check with `instanceof` (or pattern matching) before a down cast, and the ClassCastException can never happen.

Lesson: Object type casting

3.3 Inner Classes

Some objects only make sense inside another object: a heart inside a body, an entry inside a map. Inner classes let Java say exactly that.

KEY IDEAS

- **Why inner classes exist** Declare a class inside another class when the inner thing cannot exist without the outer thing. A university has departments, and a map has entries. Without a Map object there is no Map.Entry .
- **Normal inner classes** A normal inner class is declared directly inside the outer class body. Compiling produces two files: Outer.class and Outer$Inner.class .
- **Method local inner classes** A class declared inside a method exists only inside that method. Use it for a helper that is needed again and again inside one method and nowhere else.
- **Anonymous inner classes** An anonymous class has no name. You declare it and create its one object in the same expression. It is perfect for a quick, one time change of behaviour.
- **Normal class versus anonymous class**
- **Static nested classes** A nested class declared static is not tied to any outer object. Create it with new Outer.Nested() .
- **Normal inner versus static nested**
- **Classes and interfaces inside each other**

FORMULAS

Java snippet

```
class University {          // outer
 class Department { }    // inner: lives
        inside a university
                  }
```

Lesson: Why inner classes exist

Private outer members are visible

```
                class Outer {
              private int x = 10;
              static int y = 20;
                class Inner {
    void show() { System.out.println(x + " "
                + y); }
                    }
public static void main(String[] args) { new
     Outer().new Inner().show(); }
                }
```

Lesson: Normal inner classes

Effectively final locals are allowed

```
                class Main {
public static void main(String[] args) {
  int bonus = 50;                        //
   never changed: effectively final
                class Salary {
      int total(int base) { return base +
             bonus; }
                    }
          System.out.println(new
       Salary().total(1000));
                  }
                }
```

Lesson: Method local inner classes

Passed directly as an argument

```
                class Main {
public static void main(String[] args) throws
                Exception {
     Thread t = new Thread(new Runnable() {
                public void run() {
  System.out.println("defined inside the
              argument"); }
                        });
                    t.start();
                    t.join();
                  }
                }
```

Lesson: Anonymous inner classes

No instance members from a static nested class

```
class Outer {
  int x = 10;
  static class Nested {
    void show() { System.out.println(x); }
  }
}
```

Lesson: Static nested classes

Map.Entry in action

```
Map<String, Integer> marks = new TreeMap<>
    (Map.of("Asha", 91, "Ravi", 78));
for (Map.Entry<String, Integer> e :
        marks.entrySet())
    System.out.println(e.getKey() + " → " +
        e.getValue());
```

Lesson: Classes and interfaces inside each other

TABLES

Table: modifiers on a normal inner class

Modifiers on a normal inner class
`public`, default, `private`, `protected`, `final`, `abstract`, `strictfp`, and `static` (which turns it into a static nested class)

Lesson: Normal inner classes

Table: · normal class · anonymous class

	Normal class	Anonymous class
Extends a class	one	one
Implements interfaces	any number	exactly one
Extend and implement at once	yes	no, one or the other
Own constructors	yes	no, it has no name to give one
Can be reused	yes	no, one object only

Lesson: Normal class versus anonymous class

Table: · normal inner class · static nested class

	Normal inner class	Static nested class
Needs an outer object	yes	no
Can use outer instance members	yes	no, only static ones
Created with	`outer.new Inner()`	`new Outer.Nested()`
Static members inside	only since Java 16	always
Own `main` you can run	only since Java 16	always

Lesson: Normal inner versus static nested

TIPS AND TRAPS

REMEMBER — Strictly speaking, Java calls all four "nested classes" and only the non-static ones "inner classes". Most people use "inner class" loosely for all of them.

Lesson: Why inner classes exist

IN THE EXAM — A method added only in the anonymous class cannot be called through the parent reference, because the compiler only knows the parent type.

Lesson: Anonymous inner classes

REMEMBER — An anonymous class can still use an instance initializer block `{ ... }` where a constructor would normally go.

Lesson: Normal class versus anonymous class

STEP 4 Core Java APIs

4.1 java.lang: Object, String and Wrappers

Every Java program uses the java.lang package, whether it says so or not. Object is the parent of everything, String is the most used class on Earth, and wrappers let numbers behave like objects.

KEY IDEAS

- **The Object class** java.lang.Object is the root of every class. If a class extends nothing, it extends Object directly. So every object you will ever create has Object's methods.
- **toString()** println(obj) calls obj.toString() for you. Object's version returns the class name, an @ and the hash code in hexadecimal, which is rarely what a human wants.
- **hashCode()** Hash based collections such as HashMap and HashSet use the hash code to decide which bucket an object goes into, so they can find it again quickly. A good hash code spreads objects evenly.
- **equals() and ==** Object's equals does the same as == : it only checks whether both references point at the same object. Override it to compare contents .
- **clone(): shallow and deep copies** clone() makes a copy of an object. The class must implement the marker interface Cloneable , or Object's clone throws CloneNotSupportedException .
- **getClass()** getClass() returns a Class object describing the real class of an object at run time. It is the entry point to reflection.
- **String is immutable** Once a String object is created, its contents can never change. Every method that seems to change it actually returns a new String.
- **The string constant pool** String literals are stored in a special area called the string constant pool (SCP). Before creating a literal, the JVM checks the pool. If an equal String is already there, it is reused.
- **Important String methods**
- **StringBuffer** When the contents change often, String wastes objects. StringBuffer is a mutable sequence of characters: every change happens in the same object.
- **StringBuilder and choosing between the three** StringBuilder (Java 5) has exactly the same methods as StringBuffer, but none of them are synchronized. With one thread, which is almost always the case, it is faster.
- **Wrapper classes** Each primitive type has a wrapper class that holds the value as an object: Byte Short Integer Long Float Double Character Boolean . Collections and generics only store objects, so wrappers are everywhere.
- **Autoboxing and unboxing** Since Java 5 the compiler converts between primitives and wrappers automatically.
- **final versus immutable**

FORMULAS

Java snippet

```
public String toString() {
return getClass().getName() + "@" +
Integer.toHexString(hashCode());
}
```

Lesson: toString()

hashCode() Exam favourite

$$\text{hash("Ab")} = \text{'A'} \times 31 + \text{'b'} = 65 \times 31 + 98 = 2113$$

Lesson: hashCode()

hashCode()

$$\text{hash("BB")} = \text{'B'} \times 31 + \text{'B'} = 66 \times 31 + 66 = 2112$$

Lesson: hashCode()

String has a content based hash code

```
System.out.println("Ab".hashCode());
System.out.println("BB".hashCode());
System.out.println(new String("Ab").hashCode() ==
"Ab".hashCode());
```

Lesson: hashCode()

String overrides equals, StringBuilder does not

```
String s1 = new String("hi"), s2 = new
                String("hi");
StringBuilder b1 = new StringBuilder("hi"), b2 =
            new StringBuilder("hi");
        System.out.println(s1.equals(s2));
        System.out.println(b1.equals(b2));
System.out.println(b1.toString().equals(b2.toStri
                    ng()));
```

Lesson: equals() and ==

Ask an object what it really is

```
Object o = new ArrayList<String>();
      Class c = o.getClass();
    System.out.println(c.getName());
  System.out.println(c.getSimpleName());
System.out.println(c.getSuperclass().getSimpleNam
                    e());
    System.out.println(o.getClass() ==
            ArrayList.class);
```

Lesson: getClass()

The original never changes

```
        String s = new String("java");
  s.concat(" rocks");                  // a new
        object is made, then forgotten
            System.out.println(s);
s = s.concat(" rocks");              // keep the
                new object
            System.out.println(s);
 StringBuilder sb = new StringBuilder("java");
sb.append(" rocks");                 // the same
                object changes
            System.out.println(sb);
```

Lesson: String is immutable

Literals are shared, new is not

```
              String a = "java";
              String b = "java";
          String c = new String("java");
            System.out.println(a == b);
            System.out.println(a == c);
        System.out.println(a == c.intern());
```

Lesson: The string constant pool

StringBuffer

Exam favourite

$$\text{new capacity} = (\text{old capacity} + 1) \times 2$$

Lesson: StringBuffer

StringBuffer

$$16 \rightarrow (16 + 1) \times 2 = 34 \rightarrow (34 + 1) \times 2 = 70$$

Lesson: StringBuffer

StringBuffer

$$\text{new StringBuffer("java")} \rightarrow 16 + \text{"java".length()} = 20$$

Lesson: StringBuffer

Watch the capacity grow

```
      StringBuffer sb = new StringBuffer();
        System.out.println(sb.capacity());
  sb.append("abcdefghijklmnop");          // 16
            characters, still fits
        System.out.println(sb.capacity());
  sb.append("q");                          // the
                17th character
        System.out.println(sb.capacity());
sb.append("r".repeat(18));              // now 35
                  characters
        System.out.println(sb.capacity());
              System.out.println(new
         StringBuffer("java").capacity());
```

Lesson: StringBuffer

Method chaining

```
      String result = new StringBuilder()
                  .append("Hello")
                   .append(", ")
                  .append("World")
                     .reverse()
                    .toString();
          System.out.println(result);
```

Lesson: StringBuilder and choosing between the three

Same value, same object, but only for small numbers

```
      Integer a = Integer.valueOf(127), b =
              Integer.valueOf(127);
      Integer c = Integer.valueOf(128), d =
              Integer.valueOf(128);
          System.out.println(a == b);
          System.out.println(c == d);
        System.out.println(c.equals(d));
```

Lesson: Wrapper classes

Mixing freely

```
List<Integer> marks = new ArrayList<>();
marks.add(90);                          // boxing
marks.add(75);
int total = 0;
for (int m : marks) total += m;         // unboxing
Integer avg = total / marks.size(); // boxing again
System.out.println(total + " " + avg);
```

Lesson: Autoboxing and unboxing

final does not freeze the object

```
final StringBuilder sb = new StringBuilder("final");
sb.append(" but changing");
System.out.println(sb);
```

Lesson: final versus immutable

TABLES

Table: method · what it does

Method	What it does
`String toString()`	a text form of the object
`int hashCode()`	a number used by hash based collections
`boolean equals(Object o)`	are two objects equal
`Object clone()`	a copy of the object (protected)
`Class getClass()`	the runtime class of the object (final)
`void wait(), wait(long), wait(long, int)`	pause a thread until notified (final)
`void notify(), notifyAll()`	wake waiting threads (final)
`void finalize()`	old clean-up hook, deprecated since Java 9

Lesson: The Object class

Table: == · equals()

==	`equals()`
an operator, works on primitives and references	a method, works on objects only
for objects: same object or not	for objects: same contents, if the class overrides it
cannot be changed	can be overridden
unrelated types do not compile	unrelated types simply return false

Lesson: equals() and ==

Table: method · example · result

Method	Example	Result
`charAt(i)`	`"brainy".charAt(2)`	'a'
`length()`	`"brainy".length()`	6
`substring(b)`, `substring(b, e)`	`"brainy".substring(1, 4)`	"rai" (end index excluded)
`indexOf`, `lastIndexOf`	`"banana".indexOf('a')`	1 (–1 if absent)
`replace(a, b)`	`"ababa".replace('a', 'b')`	"bbbbb"
`toUpperCase()`, `toLowerCase()`	`"Java".toUpperCase()`	"JAVA"
`trim()`, `strip()`	`" hi ".trim()`	"hi" (ends only)
`equalsIgnoreCase`	`"JAVA".equalsIgnoreCase("java")`	true
`isEmpty()`, `isBlank()`	`" ".isEmpty()`, `" ".isBlank()`	false, true
`split`, `String.join`	`"a,b,c".split(",")`	["a", "b", "c"]
`repeat(n)`	`"ha".repeat(3)`	"hahaha"

Lesson: Important String methods

Table: · string · stringbuffer · stringbuilder

	String	StringBuffer	StringBuilder
Mutable	no	yes	yes
Thread safe	yes (immutable)	yes (synchronized)	no
Speed for many changes	slowest	medium	fastest
Overrides equals	yes	no	no
Use when	the text rarely changes	several threads change it	one thread changes it often

Lesson: StringBuilder and choosing between the three

Table: method · converts · example

Method	Converts	Example
`valueOf()`	primitive or String → wrapper	`Integer.valueOf("10")`, `Integer.valueOf("1111", 2)`
`xxxValue()`	wrapper → primitive	`Integer.valueOf(130).byteValue()`
`parseXxx()`	String → primitive	`Integer.parseInt("10")`, `Double.parseDouble("1.5")`
`toString()`	wrapper or primitive → String	`Integer.toString(15, 2)`, `Integer.toHexString(255)`

Lesson: Wrapper classes

Table: wrapper · cached values

Wrapper	Cached values
Byte	all (–128 to 127)
Short, Integer, Long	–128 to 127
Character	0 to 127
Boolean	TRUE and FALSE
Float, Double	none

Lesson: Wrapper classes

Table: declaration · valid · why

Declaration	Valid	Why
`long l = 10;`	yes	widening
`Long l = 10;`	no	10 is an int: it would need widening then boxing
`Long l = 10L;`	yes	boxing
`Object o = 10;`	yes	boxing then widening
`double d = 10;`	yes	widening
`Double d = 10;`	no	widening then boxing
`Number n = 10;`	yes	boxing to Integer, which is a Number

Lesson: Autoboxing and unboxing

TIPS AND TRAPS

REMEMBER The contract: if `a.equals(b)` is true, then `a.hashCode() == b.hashCode()` must be true. The reverse is not required. Break this rule and HashMap and HashSet misbehave.

Lesson: equals() and ==

REMEMBER Many teams avoid `clone()` and write a **copy constructor** instead: `new Person(other)`. It is clearer and needs no marker interface.

Lesson: clone(): shallow and deep copies

IN THE EXAM How many objects does `String s = new String("spring");` create, if the pool is empty? Two: one on the heap and one literal in the pool. Run it a second time and only one more appears, because the literal is reused.

Lesson: The string constant pool

REMEMBER When nothing actually changes, many String methods return the same object. `"java".toLowerCase() == "java"` is true, while `"java".toUpperCase()` makes a new object.

Lesson: Important String methods

REMEMBER A loop that builds text with += creates a new String every pass. Use a StringBuilder and call `toString()` once at the end.

Lesson: StringBuilder and choosing between the three

COMMON MISTAKE Never compare wrapper objects with ==. Use `equals`, or unbox them first.

Lesson: Wrapper classes

4.2 Exception Handling

Things go wrong: files vanish, networks drop, users type letters where numbers belong. Exception handling lets a program stop gracefully, or carry on with plan B, instead of crashing in a heap.

KEY IDEAS

- **What is an exception** An exception is an unwanted, unexpected event that disturbs the normal flow of a program: a file that is missing, a division by zero, a number typed as text.
- **The runtime stack and default handling** The JVM gives every thread its own runtime stack . Each method call pushes a stack frame , and each return pops it.
- **The exception hierarchy** Throwable is the root of everything that can be thrown. It has two children.
- **Checked and unchecked exceptions** The compiler checks some exceptions to make sure you handle them. These are checked exceptions. The rest are unchecked .
- **try and catch** Put the risky code in try and the handling code in catch .

- **Printing exception information**
- **finally** Clean-up code, such as closing a file or a database connection, must run whether things went well or not. Put it in finally . It runs after the try, with or without an exception, caught or not.
- **final, finally and finalize**
- **Nested try catch finally** A try block can contain another try. The inner catch handles what it can, and anything else bubbles to the outer catch. Both finally blocks always run, inner first.
- **throw** Sometimes you create an exception object yourself and hand it to the JVM. That is what throw does.
- **throws** If a method can raise a checked exception and does not want to handle it, it must say so with throws . The responsibility moves to the caller.
- **Custom exceptions** Your own problems deserve their own names, like InsufficientFundsException or TooYoungException . Create them by extending an existing exception class.
- **The top ten exceptions** Exceptions are raised either by the JVM automatically, or by a programmer or API on purpose.
- **Modern exception handling** Resources declared in the brackets after try are closed automatically, in reverse order, even if an exception happens. No finally needed. Any class that implements AutoCloseable works.

FORMULAS

Nobody catches it: the default handler takes over

```
class Main {
  static void doMoreStuff() {
    System.out.println(10 / 0); }
static void doStuff() { doMoreStuff(); }
public static void main(String[] args) {
  doStuff(); }
}
```

Lesson: The runtime stack and default handling

A checked exception must be handled or declared

```
import java.io.*;
class Main {
public static void main(String[] args) {
  PrintWriter pw = new
    PrintWriter("notes.txt");
  }
}
```

Lesson: Checked and unchecked exceptions

Parent before child

```
class Main {
public static void main(String[] args) {
  try { System.out.println(10 / 0); }
    catch (Exception e) { }
  catch (ArithmeticException e) { }
  }
}
```

Lesson: try and catch

Three levels of detail

```
try {
  System.out.println(10 / 0);
} catch (ArithmeticException e) {
  System.out.println(e);
  System.out.println(e.getMessage());
System.out.println(e.getStackTrace().length >
  0);
}
```

Lesson: Printing exception information

System.exit skips finally

```
try {
  System.out.println("try");
  System.exit(0);
} finally {
System.out.println("finally");
}
```

Lesson: finally

Nothing can follow a throw

```
throw new ArithmeticException("bye");
  System.out.println("hello");
```

Lesson: throw

Delegating to the caller

```
class Main {
  static void nap() throws InterruptedException
                    {
                  Thread.sleep(10);
              System.out.println("woke up");
                    }
  public static void main(String[] args) throws
              InterruptedException {
                      nap();
                    }
                  }
```

Lesson: throws

StackOverflowError

```
class Main {
  static void forever() { forever(); }
public static void main(String[] args) {
              forever(); }
                    }
```

Lesson: The top ten exceptions

Parent and child in one multi-catch

```
      try { System.out.println(10 / 0); }
catch (ArithmeticException | RuntimeException e)
                    { }
```

Lesson: Modern exception handling

TABLES

Table: exception · kind

Exception	Kind
IOException	fully checked
RuntimeException	unchecked
InterruptedException	fully checked
Error	unchecked
Throwable	partially checked
ArithmeticException	unchecked
Exception	partially checked
FileNotFoundException	fully checked

Lesson: Checked and unchecked exceptions

Table: case · runs · ending

Case	Runs	Ending
No exception	1, 2, 3, 5	normal
Exception at stmt2, catch matches	1, 4, 5	normal
Exception at stmt2, catch does not match	1	abnormal
Exception at stmt4 or stmt5	whatever ran before it	always abnormal

Lesson: try and catch

Table: method · prints

Method	Prints
`e.printStackTrace()`	name: description, then the stack trace (this is what the default handler uses)
`e.toString()` or `println(e)`	name: description
`e.getMessage()`	description only

Lesson: Printing exception information

Table: word · what it is · purpose

Word	What it is	Purpose
`final`	a modifier	no extending, no overriding, no reassigning
`finally`	a block after try	clean-up related to a try
`finalize()`	a method of Object	clean-up the garbage collector once called before destroying an object. Deprecated since Java 9, do not use it

Lesson: final, finally and finalize

Table: keyword · meaning

Keyword	Meaning
`try`	the risky code
`catch`	the handling code
`finally`	the clean-up code
`throw`	hand an exception object to the JVM yourself
`throws`	pass the responsibility to the caller

Lesson: throws

Table: exception · raised by · when

Exception	Raised by	When
ArrayIndexOutOfBoundsException	JVM	an array index outside 0 to length − 1
NullPointerException	JVM	calling something on null
ClassCastException	JVM	casting an object to a type it is not
StackOverflowError	JVM	recursion that never stops
NoClassDefFoundError	JVM	a .class file that was there at compile time is missing at run time
ExceptionInInitializerError	JVM	a crash inside a static block or static initialiser
IllegalArgumentException	programmer or API	a method receives an argument it cannot accept
NumberFormatException	programmer or API	text that is not a number is parsed (a child of IllegalArgumentException)
IllegalStateException	programmer or API	a method is called at the wrong time
AssertionError	programmer	an assert statement fails

Lesson: The top ten exceptions

TIPS AND TRAPS

REMEMBER When a program has several threads, only the thread that failed stops. The others keep running.

Lesson: The runtime stack and default handling

REMEMBER `RuntimeException`, `Error` and all their children are unchecked. Everything else is checked.

Lesson: Checked and unchecked exceptions

IN THE EXAM Whether an exception is checked or unchecked is decided by the compiler, but **every** exception happens at run time. "Checked" never means "happens at compile time".

Lesson: Checked and unchecked exceptions

COMMON MISTAKE Returning from finally hides exceptions and confuses readers. Use finally for clean-up only.

Lesson: finally

IN THE EXAM The only thing that stops finally is the JVM itself shutting down, for example `System.exit(0)`.

Lesson: finally

REMEMBER finally is preferred over finalize because you know exactly when it runs. Nobody can predict when, or even whether, the garbage collector calls finalize.

Lesson: final, finally and finalize

REMEMBER try must be followed by catch or finally (or both). catch and finally cannot appear without a try. A try with resources may stand alone.

Lesson: Nested try catch finally

REMEMBER If a try block can never throw a **fully checked** exception, a catch for it does not compile. Catching `Exception`, `Throwable` or any unchecked exception is always allowed.

Lesson: throws

4.3 Enum

Some values come from a fixed menu: days of the week, traffic light colours, coffee sizes. An enum turns that menu into a type, so nobody can order a size that does not exist.

KEY IDEAS

- **Why enums** An enum is a type with a fixed list of named values. It arrived in Java 5.
- **Declaring and using an enum**
- **Enums in switch** Enums have been allowed in switch since Java 5. Inside the cases, write the constant name without the enum name.
- **java.lang.Enum** Every enum is automatically a child of java.lang.Enum . Since a class can extend only one class, an enum cannot extend anything else, and nobody can extend an enum.
- **values(), ordinal() and valueOf()**
- **Constructors, fields and methods** An enum can hold data. Give it fields and a constructor, and pass values in brackets after each constant.
- **Constant-specific bodies** Each constant can override a method with its own body. Declare the method abstract in the enum and every constant must provide it.
- **An enum with main** An enum is a class, so it can have a main method and be run directly. Remember that constants must come first.
- **enum, Enum and Enumeration**
- **EnumSet and EnumMap** Java has collections built specially for enums. They are faster and smaller than general purpose ones, because they use the ordinal internally.

FORMULAS

A traffic light

```
enum Light { RED, AMBER, GREEN }
class Main {
public static void main(String[] args) {
Light now = Light.GREEN;
System.out.println(now);
System.out.println(now == Light.GREEN);
}
}
```

Lesson: Declaring and using an enum

Every label must be a constant of that enum

```
enum Light { RED, AMBER, GREEN }
class Main {
public static void main(String[] args) {
Light l = Light.RED;
switch (l) { case RED: break; case BLUE:
break; }
}
}
```

Lesson: Enums in switch

An enum cannot extend a class

```
class Base { }
enum Light extends Base { RED }
```

Lesson: java.lang.Enum

Walking through the week

```
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }
class Main {
public static void main(String[] args) {
for (Day d : Day.values())
System.out.print(d + ":" + d.ordinal() + " ");
System.out.println();
Day d = Day.valueOf("FRI");
System.out.println(d.compareTo(Day.MON) +
" " + Day.values().length);
}
}
```

Lesson: values(), ordinal() and valueOf()

No new for enums

```
enum Coffee { SMALL; }
class Main {
public static void main(String[] args) {
Coffee c = new Coffee(); }
}
```

Lesson: Constructors, fields and methods

Running an enum

```
enum Fish {
STAR, GUPPY;
public static void main(String[] args) {
System.out.println("Enum main, " +
Fish.values().length + " fish");
}
}
```

Lesson: An enum with main

TABLES

Table: method · gives

Method	Gives
`Day.values()`	an array of all constants, in declaration order (added by the compiler)
`d.ordinal()`	the position of the constant, starting at 0
`d.name()`	the exact name as written
`Day.valueOf("MON")`	the constant with that exact name
`a.compareTo(b)`	the difference of their ordinals

Lesson: values(), ordinal() and valueOf()

Table: name · what it is · used for

Name	What it is	Used for
`enum`	a keyword	declaring a group of named constants
`Enum`	a class in java.lang	the parent of every enum
`Enumeration`	an interface in java.util	an old cursor for walking through Vector and Hashtable (see Collections)

Lesson: enum, Enum and Enumeration

TIPS AND TRAPS

REMEMBER — A switch **expression** over an enum that lists every constant needs no default branch. If someone later adds a constant, the compiler points at every switch that forgot it.

Lesson: Enums in switch

COMMON MISTAKE — Do not store ordinals in a database. Adding a constant in the middle changes every number after it. Store the name instead.

Lesson: values(), ordinal() and valueOf()

REMEMBER Constant-specific bodies replace long switch statements. Adding a new operation means adding one constant, not hunting down every switch.

Lesson: Constant-specific bodies

4.4 Regular Expressions

A regular expression is a tiny language for describing text patterns: ten digits, a word starting with a capital, anything that looks like an email. Learn it once and use it in Java, testing tools and every editor.

KEY IDEAS

- **What is a regular expression** A regular expression (regex) describes a group of strings that follow one pattern. For example, [6-9][0-9]{9} describes every Indian mobile number.
- **Pattern and Matcher** Java's regex classes live in java.util.regex .
- **Character classes** Square brackets match one character from a set.
- **Predefined character classes**
- **Quantifiers** Quantifiers say how many times something may repeat.
- **Splitting text** Both Pattern and String can split text around a regex.
- **Validating input** matches() checks whether the whole string fits the pattern, unlike find() , which looks for a piece of it.
- **Find, extract and replace** Round brackets create capturing groups , which you can read with group(1) , group(2) and so on, or reuse in a replacement as $1 .

FORMULAS

Find every ab

```
Pattern p = Pattern.compile("ab");
Matcher m = p.matcher("abbbabbaba");
int count = 0;
while (m.find()) {
count++;
System.out.println(m.start() + "..." +
m.end() + " " + m.group());
}
System.out.println("matches: " + count);
```

Lesson: Pattern and Matcher

Predefined classes in action

```
String t = "a7b k@9_";
for (String regex : new String[]{"\\s", "\\d",
"\\D", "\\w", "\\W", "."}) {
Matcher m =
Pattern.compile(regex).matcher(t);
int n = 0;
while (m.find()) n++;
System.out.println(regex + " matches " + n +
" characters");
}
```

Lesson: Predefined character classes

StringTokenizer

```
StringTokenizer st = new StringTokenizer("19-10-
2026", "-");
System.out.println(st.countTokens());
while (st.hasMoreTokens())
System.out.print(st.nextToken() + " ");
```

Lesson: Splitting text

Validating input Exam favourite

[a-k][a-zA-Z0-9#]+

Lesson: Validating input

Identifier checker

```
String regex = "[a-k][a-zA-Z0-9#]+";
for (String id : new String[]{"apple", "kid#7",
"z99", "b", "c-d"})
System.out.println(id + " → " +
id.matches(regex));
```

Lesson: Validating input

Tidy up spaces and add up every number

```
String messy = "  apples 12   bananas 5  cherries
30 ";
System.out.println("[" +
messy.trim().replaceAll("\\s+", " ") + "]");
Matcher m =
Pattern.compile("\\d+").matcher(messy);
int total = 0;
while (m.find()) total +=
Integer.parseInt(m.group());
System.out.println("total fruit: " + total);
```

Lesson: Find, extract and replace

TABLES

Table: class · matches one

Class	Matches one
`[abc]`	a, b or c
`[^abc]`	any character except a, b and c
`[a-z]`	any lowercase letter
`[A-Z]`	any uppercase letter
`[a-zA-Z]`	any letter
`[0-9]`	any digit
`[a-zA-Z0-9]`	any letter or digit
`[^a-zA-Z0-9]`	anything that is not a letter or digit (a special character)

Lesson: Character classes

Table: class · same as · matches one

Class	Same as	Matches one
`\s`		whitespace (space, tab, new line)
`\S`		anything except whitespace
`\d`	`[0-9]`	a digit
`\D`	`[^0-9]`	anything except a digit
`\w`	`[a-zA-Z0-9_]`	a word character
`\W`	`[^a-zA-Z0-9_]`	anything except a word character
`.`		any character (except a line break)

Lesson: Predefined character classes

Table: quantifier · means

Quantifier	Means
`a`	exactly one a
`a+`	one or more a
`a*`	zero or more a
`a?`	zero or one a
`a{3}`	exactly three a
`a{2,4}`	two to four a
`a{2,}`	two or more a

Lesson: Quantifiers

TIPS AND TRAPS

REMEMBER — In Java source code, a backslash inside a String must be doubled. The regex `\d` is written `"\\d"`.

Lesson: What is a regular expression

IN THE EXAM — A literal dot must be escaped: `"\\."`. Without the backslash, `.` matches every character, which is a classic bug when splitting on dots.

Lesson: Predefined character classes

REMEMBER Quantifiers are **greedy**: they grab as much as they can. Add ? to make them lazy: <.+?> matches one HTML tag instead of everything from the first < to the last >.

Lesson: Quantifiers

COMMON MISTAKE Real email rules are very complicated. For production, use a well tested library and confirm the address by sending a mail.

Lesson: Validating input

REMEMBER If you use the same regex many times, compile it once into a `Pattern` and reuse it. `String.matches` compiles the pattern again on every call.

Lesson: Find, extract and replace

4.5 Internationalization (I18N)

India writes 12,34,567.89, America writes 1,234,567.89 and Germany writes 1.234.567,89. Internationalization lets one program speak to all of them correctly, without rewriting a line of logic.

KEY IDEAS

- **What is I18N** Internationalization (shortened to I18N, because there are 18 letters between the I and the N) means designing a program so it can adapt to many countries: their number formats, currencies, dates and languages.
- **Locale** A java.util.Locale object describes a language and, optionally, a country. It is final , Cloneable and Serializable .
- **NumberFormat** java.text.NumberFormat is abstract. Ask its factory methods for an object, then call format to turn a number into text, or parse to go the other way.
- **Controlling digits** A NumberFormat can be told how many digits to show before and after the decimal point.
- **DateFormat** java.text.DateFormat formats an old style java.util.Date . Pick a style and a locale.
- **Modern dates with java.time**
- **ResourceBundle: translated messages** Text shown to users should never be hard coded. Put each language's messages in its own resource bundle and let Java pick the right one for the locale. Bundles are usually .properties files, such as Messages_hi.properties .

FORMULAS

One number, four countries

```
        double amount = 1234567.891;
    Locale[] places = { Locale.of("en", "IN"),
      Locale.US, Locale.UK, Locale.GERMANY };
           for (Locale l : places) {
                NumberFormat num =
          NumberFormat.getInstance(l);
                NumberFormat cur =
       NumberFormat.getCurrencyInstance(l);
           System.out.println(l + "\t" +
 num.format(amount) + "\t" + cur.format(amount));
                       }
```

Lesson: NumberFormat

Banker's rounding

```
                NumberFormat f =
        NumberFormat.getInstance(Locale.US);
           f.setMaximumFractionDigits(0);
      System.out.println(f.format(2.5) + " " +
       f.format(3.5) + " " + f.format(2.51));
```

Lesson: Controlling digits

TABLES

Table: factory method · for

Factory method	For
`getInstance(locale)` or `getNumberInstance(locale)`	plain numbers
`getCurrencyInstance(locale)`	money
`getPercentInstance(locale)`	percentages

Lesson: NumberFormat

Table: method · effect

Method	Effect
`setMaximumFractionDigits(n)`	round to at most n decimals
`setMinimumFractionDigits(n)`	pad with zeros to at least n decimals
`setMaximumIntegerDigits(n)`	keep only the last n digits before the point
`setMinimumIntegerDigits(n)`	pad with zeros in front

Lesson: Controlling digits

Table: style · constant value

Style	Constant value
`DateFormat.FULL`	0
`DateFormat.LONG`	1
`DateFormat.MEDIUM`	2 (the default)
`DateFormat.SHORT`	3

Lesson: DateFormat

TIPS AND TRAPS

REMEMBER `Locale.getDefault()` returns the locale of the machine, and `Locale.getAvailableLocales()` lists every locale the JVM supports.

Lesson: Locale

COMMON MISTAKE `parse` throws the checked `ParseException` when the text does not fit the format, so it must be handled or declared.

Lesson: NumberFormat

IN THE EXAM NumberFormat rounds with **HALF_EVEN** (banker's rounding) by default. So with 0 decimals, 2.5 becomes 2 and 3.5 becomes 4.

Lesson: Controlling digits

REMEMBER `DateFormat` objects are not thread safe. Do not share one between threads, or switch to the modern `java.time` classes below, which are.

Lesson: DateFormat

STEP 5 Collections and Generics

5.1 Collections Framework

Arrays are fixed boxes. Collections are smart containers that grow, sort, search and remove duplicates for you. Choosing the right one is half of writing fast Java.

KEY IDEAS

- **Arrays versus collections** Arrays have three big limits: their size is fixed, they hold only one type, and they come with almost no ready-made methods. Collections fix all three.
- **The nine key interfaces** add , addAll , remove , removeAll , retainAll , clear , contains , containsAll , isEmpty , size , toArray and iterator . Collection has no method to get an element by position, because not every collection has positions.
- **The List interface** A List keeps elements in insertion order, gives every element an index, and allows duplicates and nulls.
- **ArrayList** ArrayList is a resizable array . It is the most used collection in Java.
- **LinkedList** LinkedList is a doubly linked list : each element knows the one before it and the one after it. It implements both List and Deque .
- **Vector and Stack** Vector (Java 1.0) is an older resizable array whose methods are all synchronized. Stack extends Vector and works last in, first out . Both are called legacy classes.
- **Cursors: Enumeration, Iterator, ListIterator** A cursor walks through a collection one element at a time.
- **Set, HashSet, LinkedHashSet and TreeSet** A Set never holds duplicates. Adding one again simply returns false .
- **Comparable and Comparator** Sorting needs a way to compare two objects. Java offers two.
- **Map and its classes** A Map stores key → value pairs. Keys are unique, values may repeat. Each pair is a Map.Entry . Map is not a child of Collection.
- **NavigableSet and NavigableMap** Java 6 added navigation methods to TreeSet and TreeMap, answering questions like "the closest value at or below this one".
- **Queue and PriorityQueue** A Queue holds elements waiting to be processed, usually first in, first out.
- **The Collections and Arrays classes** binarySearch needs a sorted list. If the key is missing it returns −(insertion point) − 1 . Above, "J" would go at index 1, so the result is −1 − 1 = −2.
- **Modern collection tools**

FORMULAS

A collection grows by itself

```
List<String> guests = new ArrayList<>();
guests.add("Asha");
guests.add("Ravi");
guests.add("Meera");          // no size planning needed
guests.remove("Ravi");
System.out.println(guests + " size " + guests.size());
```

Lesson: Arrays versus collections

Bulk operations

```
List<Integer> a = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6));
a.removeAll(List.of(2, 4));            // remove these
System.out.println(a);
a.retainAll(List.of(1, 5, 6, 9));      // keep only these
System.out.println(a + " " + a.containsAll(List.of(1, 6)));
```

Lesson: The nine key interfaces

Index based methods

```
List<String> l = new ArrayList<>(List.of("a", "b", "c", "b"));
l.add(1, "X");                         // insert at index 1
l.set(0, "A");                         // replace index 0
System.out.println(l + " " + l.get(2) + " " + l.indexOf("b") + " " + l.lastIndexOf("b"));
l.remove(0);                           // remove by index
l.remove("b");                         // remove the first "b" by value
System.out.println(l);
```

Lesson: The List interface

Common ArrayList moves

```
ArrayList<Object> l = new ArrayList<>();
l.add("A"); l.add(10); l.add("A"); l.add(null);
System.out.println(l);
l.remove(2);
l.add(2, "M");
l.add("N");
System.out.println(l + " " + (l instanceof RandomAccess));
```

Lesson: ArrayList

Working at both ends

```
LinkedList<String> l = new LinkedList<>
          (List.of("B", "C"));
            l.addFirst("A");
            l.addLast("D");
System.out.println(l + " first " + l.getFirst() +
        " last " + l.getLast());
            l.removeFirst();
            l.removeLast();
         System.out.println(l);
```

Lesson: LinkedList

A stack of plates

```
Stack<String> plates = new Stack<>();
 plates.push("A"); plates.push("B");
          plates.push("C");
      System.out.println(plates);
System.out.println(plates.pop() + " " +
           plates.peek());
          plates.push("D");
System.out.println(plates.search("A") + " " +
          plates.search("Z"));
```

Lesson: Vector and Stack

Remove odd numbers safely with an Iterator

```
List<Integer> nums = new ArrayList<>(List.of(1,
            2, 3, 4, 5, 6));
   Iterator<Integer> it = nums.iterator();
 while (it.hasNext()) if (it.next() % 2 == 0)
              it.remove();
        System.out.println(nums);
```

Lesson: Cursors: Enumeration, Iterator, ListIterator

Three sets, same input

```
List<String> input = List.of("mango", "apple",
        "kiwi", "apple", "banana");
  System.out.println(new HashSet<>(input));
System.out.println(new LinkedHashSet<>(input));
  System.out.println(new TreeSet<>(input));
     Set<String> s = new HashSet<>();
   System.out.println(s.add("x") + " " +
              s.add("x"));
```

Lesson: Set, HashSet, LinkedHashSet and TreeSet

compareTo on Strings

```
     System.out.println("A".compareTo("Z"));
     System.out.println("Z".compareTo("K"));
     System.out.println("A".compareTo("A"));
System.out.println("apple".compareTo("apricot"));
```

Lesson: Comparable and Comparator

LinkedHashMap keeps insertion order

```
Map<String, Integer> lhm = new LinkedHashMap<>();
   lhm.put("zebra", 1); lhm.put("apple", 2);
            lhm.put("mango", 3);
          System.out.println(lhm);
lhm.merge("apple", 10, Integer::sum);       //
     modern helper: add to the old value
          System.out.println(lhm);
```

Lesson: Map and its classes

Unmodifiable views reject changes

```
      List<String> base = new ArrayList<>
          (List.of("a"));
        List<String> ro =
    Collections.unmodifiableList(base);
            ro.add("b");
```

Lesson: The Collections and Arrays classes

List.of is read-only

```
      List<String> l = List.of("a", "b");
            l.add("c");
```

Lesson: Modern collection tools

TABLES

Table: · arrays · collections

	Arrays	Collections
Size	fixed	grows and shrinks
Memory	can waste unused slots	uses what it needs
Speed	fastest raw access	slightly slower
Contents	one type only	any objects (generics keep one type safely)
Primitives	yes	only as wrapper objects
Ready methods	very few	sorting, searching, removing duplicates and many more

Lesson: Arrays versus collections

Table: · arraylist · linkedlist · vector

	ArrayList	LinkedList	Vector
Structure	resizable array	doubly linked list	resizable array
Synchronized	no	no	yes
get(i)	fast	slow	fast
Insert in the middle	slow	fast once positioned	slow
Introduced	Java 1.2	Java 1.2	Java 1.0 (legacy)

Lesson: Vector and Stack

Table: · enumeration · iterator · listiterator

	Enumeration	Iterator	ListIterator
Works with	legacy classes only (Vector, Hashtable)	every collection	Lists only
Direction	forward	forward	forward and backward
Can do	read	read, remove	read, remove, replace, add
Get one with	`v.elements()`	`c.iterator()`	`l.listIterator()`
Methods	hasMoreElements, nextElement	hasNext, next, remove	plus hasPrevious, previous, nextIndex, previousIndex, set, add

Lesson: Cursors: Enumeration, Iterator, ListIterator

Table: · hashset · linkedhashset · treeset

	HashSet	LinkedHashSet	TreeSet
Structure	hash table	hash table + linked list	balanced tree (red-black)
Order	no guaranteed order	insertion order	sorted
null	one allowed	one allowed	not allowed
Elements must be	anything	anything	mutually comparable (or use a Comparator)
add, remove, contains	constant time on average	constant time on average	logarithmic time

Lesson: Set, HashSet, LinkedHashSet and TreeSet

Table: class · order · null keys · synchronized

Class	Order	null keys	Synchronized	Notes
HashMap	none	one	no	the everyday choice
LinkedHashMap	insertion (or access) order	one	no	great for caches
TreeMap	sorted by key	none	no	NavigableMap methods
Hashtable	none	none (no null values either)	yes	legacy
IdentityHashMap	none	one	no	compares keys with == instead of equals
WeakHashMap	none	one	no	entries vanish once the key is garbage collected

Lesson: Map and its classes

Table: method · returns

Method	Returns
`floor(e)`	the largest element ≤ e
`lower(e)`	the largest element < e
`ceiling(e)`	the smallest element ≥ e
`higher(e)`	the smallest element > e
`pollFirst()`, `pollLast()`	remove and return the first or last
`descendingSet()`	a reversed view

Lesson: NavigableSet and NavigableMap

Table: action · returns a special value · throws an exception

Action	Returns a special value	Throws an exception
insert	`offer(e)` → false	`add(e)`
remove the head	`poll()` → null	`remove()`
look at the head	`peek()` → null	`element()`

Lesson: Queue and PriorityQueue

TIPS AND TRAPS

IN THE EXAM — With a `List<Integer>`, `remove(1)` removes the element at **index** 1. To remove the value 1, write `remove(Integer.valueOf(1))`.

Lesson: The List interface

REMEMBER — Best choice when you mostly **read by index** or add at the end. If you constantly insert and delete in the middle, consider LinkedList or a different design.

Lesson: ArrayList

COMMON MISTAKE — Changing a collection directly while a for-each loop or iterator walks over it throws `ConcurrentModificationException`. Iterators are **fail-fast**. Use the iterator's own `remove`, or `removeIf`.

Lesson: Cursors: Enumeration, Iterator, ListIterator

REMEMBER — A LinkedHashSet is ideal for a cache or a "recently seen" list: no duplicates, and you still know the order things arrived.

Lesson: Set, HashSet, LinkedHashSet and TreeSet

REMEMBER — `compare(a, b)` and `a.compareTo(b)` return a negative number if a comes first, a positive number if b comes first, and 0 if they are equal in that order.

Lesson: Comparable and Comparator

IN THE EXAM — If a TreeSet's comparator returns 0 for two different objects, the set treats them as duplicates and keeps only the first. Above, "kiwi" and "date" have the same length, which is why the second comparison step matters.

Lesson: Comparable and Comparator

REMEMBER — `ArrayDeque` is the fastest general purpose queue and stack in Java. It works at both ends: `offerFirst`, `offerLast`, `pollFirst`, `pollLast`.

Lesson: Queue and PriorityQueue

REMEMBER — binarySearch needs a sorted list. If the key is missing it returns `(-(insertion point) - 1)`. Above, "J" would go at index 1, so the result is −1 − 1 = −2.

Lesson: The Collections and Arrays classes

REMEMBER When many threads share a map, use `ConcurrentHashMap`. It allows safe reads and writes from many threads at once, without locking the whole map like Hashtable does.

Lesson: Modern collection tools

5.2 Generics

Generics let you say "this list holds only Strings" once, and then the compiler checks every add and every get for you. No casts, no surprise ClassCastException at midnight.

KEY IDEAS

- **Why generics** Generics (Java 5) solve two problems that old collections had.
- **Generic classes** A generic class declares a type parameter in angle brackets. The caller fills it in.
- **Bounded types** Sometimes only certain types make sense. <T extends Number> allows Number or any of its children, and lets you call Number methods on T.
- **Generic methods** A single method can be generic too. Declare the type parameter just before the return type .
- **Wildcards: ?, ? extends and ? super** A wildcard ? stands for "some type I do not know". It is used in parameters and variables, not when creating objects.
- **Generics and polymorphism**
- **Type erasure** Generics exist only for the compiler. After checking, it erases the type arguments, so at run time an ArrayList<String> and an ArrayList<Integer> are the same class.
- **Raw types and legacy code** A generic class used without type arguments, like plain ArrayList , is a raw type . Raw types exist so that pre Java 5 code still compiles, and the compiler only warns about them.

FORMULAS

Without generics: compiles, then crashes

```
ArrayList names = new ArrayList();       // raw
              type: holds anything
               names.add("Asha");
names.add(10);                          // nobody
                  stops this
            for (Object o : names) {
    String s = (String) o;               // cast
          needed, and it fails for 10
       System.out.println(s.toUpperCase());
                    }
```

Lesson: Why generics

String is not a Number

```
class Stats<T extends Number> { }
                class Main {
public static void main(String[] args) {
 Stats<String> s = new Stats<>(); }
                    }
```

Lesson: Bounded types

Cannot add to ? extends

```
Listextends Number> nums = new ArrayList<Integer>
                    ();
                nums.add(10);
```

Lesson: Wildcards: ?, ? extends and ? super

Base type can change

```
    List<String> l1 = new ArrayList<String>();
            // ArrayList IS-A List
   Collection<String> l2 = new LinkedList<>();
System.out.println(l1.getClass().getSimpleName()
     + " " + l2.getClass().getSimpleName());
```

Lesson: Generics and polymorphism

Same class at run time

```
       List<String> a = new ArrayList<>();
      List<Integer> b = new ArrayList<>();
System.out.println(a.getClass() == b.getClass());
   System.out.println(a.getClass().getName());
```

Lesson: Type erasure

TABLES

Table: parameter type · accepts · can read as · can add

Parameter type	Accepts	Can read as	Can add
`List<?>`	a list of anything	Object	only null
`List<? extends Number>`	List of Number, Integer, Double...	Number	only null
`List<? super Integer>`	List of Integer, Number, Object	Object	Integer values

Lesson: Wildcards: ?, ? extends and ? super

TIPS AND TRAPS

REMEMBER The compiler infers T from the arguments. You rarely need to write `Main.<String>firstOrDefault(...)` by hand.

Lesson: Generic methods

REMEMBER **PECS**: producer `extends`, consumer `super`. If a list gives you values, use `? extends`. If you put values into it, use `? super`.

Lesson: Wildcards: ?, ? extends and ? super

COMMON MISTAKE Never ignore "unchecked" warnings. Wrap risky legacy calls, or check the contents before trusting them.

Lesson: Raw types and legacy code

STEP 6 Multithreading

6.1 Multithreading

A thread is an independent path of execution. With several threads, one program can download, compute and respond to the user at the same time. The power is real, and so are the traps.

KEY IDEAS

- **Multitasking** Multitasking means doing several tasks at the same time. It comes in two flavours.
- **Defining a thread by extending Thread** Which line prints first is decided by the thread scheduler , a part of the JVM and operating system. Its algorithm is not fixed, so the exact order can change from run to run.
- **Defining a thread with Runnable** The second way: implement Runnable (a functional interface with one method, run()), and hand it to a Thread .
- **Names and priorities** Every thread has a name: one you give, or a default like Thread-0 . The thread that runs main is called main .
- **The thread life cycle**
- **yield, join and sleep**
- **Interrupting a thread** One thread can interrupt another that is sleeping or waiting by calling t.interrupt() . The sleeping thread wakes up with an InterruptedException . If the target is not sleeping yet, the request is remembered and fires the next time it sleeps or waits.
- **Synchronization** When several threads change the same data at once, updates can be lost. This is a race condition . count++ looks like one step but is three: read, add, write.
- **Inter-thread communication: wait and notify** Threads can signal each other. A thread calls wait() on an object to pause until another thread calls notify() or notifyAll() on the same object.
- **Deadlock** A deadlock happens when two threads wait for each other forever. Thread 1 holds lock A and wants lock B. Thread 2 holds lock B and wants lock A. Neither can move.
- **Daemon threads** A daemon thread runs in the background to support other threads. The garbage collector is the famous example. When the last non-daemon thread ends, the JVM stops all daemon threads and exits.

- **Modern concurrency** Creating threads by hand is fine for learning. Real programs use the java.util.concurrent package.

FORMULAS

Starting twice

```
Thread t = new Thread(() -> { });
t.start();
t.start();
```

Lesson: Defining a thread by extending Thread

Names

```
class Main {
  public static void main(String[] args) {

System.out.println(Thread.currentThread().getName
());
    Thread t = new Thread(() -> { });
    System.out.println(t.getName());
  Thread.currentThread().setName("boss");

System.out.println(Thread.currentThread().getName
());
    }
  }
```

Lesson: Names and priorities

Java snippet

```
NEW ──start()──▶ RUNNABLE ──run() ends──▶
TERMINATED
                          │ ▲
sleep, join,      │ │ time over, notified,
wait, lock busy   ▼ │ joined thread ended,
lock free
TIMED_WAITING / WAITING / BLOCKED
```

Lesson: The thread life cycle

sleep: a slide show

```
class Main {
  public static void main(String[] args) throws
    InterruptedException {
    for (String slide : new String[]{"intro",
      "demo", "thanks"}) {
      System.out.println("showing " +
        slide);
      Thread.sleep(100);
      }
    }
  }
```

Lesson: yield, join and sleep

A race: two threads, lost updates

```
class Main {
  static int count = 0;
public static void main(String[] args) throws
    InterruptedException {
  Runnable job = () -> { for (int i = 0; i
    < 1_000_000; i++) count++; };
  Thread a = new Thread(job), b = new
    Thread(job);
  a.start(); b.start();
  a.join(); b.join();
System.out.println("expected 2000000, got
  " + count);
    }
  }
```

Lesson: Synchronization

wait outside synchronized

```
Object lock = new Object();
lock.wait();
```

Lesson: Inter-thread communication: wait and notify

Do not run: it hangs forever

```
Object a = new Object(), b = new Object();
      Thread t1 = new Thread(() -> {
 synchronized (a) { pause(); synchronized (b)
   { System.out.println("t1 done"); } }
                    });
      Thread t2 = new Thread(() -> {
 synchronized (b) { pause(); synchronized (a)
   { System.out.println("t2 done"); } }
                    });
t1.start(); t2.start();      // each grabs one
        lock, then waits for the other
```

Lesson: Deadlock

TABLES

Table: · yield() · join() · sleep(ms)

	`yield()`	`join()`	`sleep(ms)`
Purpose	offer the processor to other threads of the same priority	wait until another thread finishes	pause for some time
Static	yes	no (called on the thread to wait for)	yes
Overloaded	no	yes, with a timeout	yes
Throws InterruptedException	no	yes	yes
Native	yes	no	the one-argument version

Lesson: yield, join and sleep

Table: method · releases the lock

Method	Releases the lock
yield()	no
join()	no
sleep()	no
wait()	yes
notify(), notifyAll()	yes, after the synchronized block ends

Lesson: Inter-thread communication: wait and notify

TIPS AND TRAPS

IN THE EXAM — A thread can be started only once. Calling `start()` a second time throws `IllegalThreadStateException`.

Lesson: Defining a thread by extending Thread

REMEMBER — Prefer `Runnable`. A class that extends Thread can never extend anything else, while a Runnable class still can. It also separates the job from the worker that runs it.

Lesson: Defining a thread with Runnable

COMMON MISTAKE — Priorities are only hints. Many operating systems ignore them, so never rely on priority for correctness.

Lesson: Names and priorities

REMEMBER `yield()` is rarely useful in practice, and the scheduler may ignore it. It is mostly an exam topic today.

Lesson: yield, join and sleep

REMEMBER Waiting for locks costs time. Keep synchronized regions small, or use ready-made thread safe tools such as `AtomicInteger` and `ConcurrentHashMap`.

Lesson: Synchronization

STEP 7 Files and Serialization

7.1 File I/O

Programs forget everything when they stop, unless they write it down. File I/O is how Java saves data to disk and reads it back, from a single word to a million lines.

KEY IDEAS

- **The File class** A java.io.File object represents a path to a file or folder. Creating the object does not create anything on disk.
- **FileWriter** FileWriter writes characters to a file. By default it overwrites the file. Pass true as a second argument to append instead.
- **FileReader** FileReader reads characters. read() returns the next character as an int , or −1 at the end of the file.
- **BufferedWriter and BufferedReader** Buffered classes collect characters in memory and move them in big chunks, which is much faster. They also understand lines .
- **PrintWriter** PrintWriter is the most convenient writer. It can wrap a file name, a File or another Writer, and its print , println and printf accept any type, just like System.out .
- **Classic file programs** These small programs combine the classes above. They are common interview and lab exercises.
- **Byte streams and character streams**
- **Modern file handling with java.nio**

FORMULAS

A File object is only a name until you act

```
import java.io.*;
class Main {
public static void main(String[] args) throws
IOException {
File f = new File("notes.txt");
System.out.println(f.exists());
System.out.println(f.createNewFile());
// now it really exists
System.out.println(f.exists() + " " +
f.isFile() + " " + f.length());
System.out.println(f.createNewFile());
// already there, so false
}
}
```

Lesson: The File class

BufferedWriter needs a Writer, not a file name

```
import java.io.*;
class Main {
public static void main(String[] args) throws
IOException {
BufferedWriter bw = new
BufferedWriter("cities.txt");
}
}
```

Lesson: BufferedWriter and BufferedReader

TABLES

Table: method · does

Method	Does
`exists()`	is there something at this path
`createNewFile()`	creates an empty file, false if it already exists
`mkdir()`, `mkdirs()`	creates a folder (mkdirs also creates missing parents)
`isFile()`, `isDirectory()`	what kind of thing is it
`list()`, `listFiles()`	names or File objects inside a folder
`length()`	file size in bytes
`delete()`	removes a file or an empty folder

Lesson: The File class

Table: class · best for

Class	Best for
FileWriter / FileReader	the simplest character access
BufferedWriter / BufferedReader	fast, line by line work
PrintWriter	writing any type of value with print, println and printf

Lesson: PrintWriter

TIPS AND TRAPS

REMEMBER In Java, a **directory** is a File too. The same class handles both.
Lesson: The File class

COMMON MISTAKE Forgetting `close()` (or `flush()`) can leave the last characters unwritten. Use try-with-resources so files always close.
Lesson: FileWriter

REMEMBER Closing the outer BufferedWriter or BufferedReader automatically closes the FileWriter or FileReader inside it.
Lesson: BufferedWriter and BufferedReader

IN THE EXAM `write(100)` writes the character d. `print(100)` writes the three characters `100`.
Lesson: PrintWriter

REMEMBER Never copy an image with FileReader and FileWriter. They try to decode bytes as text and will damage binary data.
Lesson: Byte streams and character streams

REMEMBER Text files should state their encoding. `Files` methods use UTF-8 by default, and since Java 18 so does FileReader and FileWriter, so Hindi, Kannada, Odia and Arabic text survive the round trip.
Lesson: Modern file handling with java.nio

7.2 Serialization

Serialization turns a live Java object into a stream of bytes that can go to a file or across a network. Deserialization rebuilds the object from those bytes. One interface and two streams do the work, and a handful of rules decide what survives the trip.

KEY IDEAS

- **What serialization means** An object lives in memory. When the program ends, it is gone. Serialization is the process of writing an object's state as bytes, so it can be stored in a file or sent over a network.
- **Writing and reading an object** The loaded object has the same state , but it is a brand new object, so p == q is false.
- **The transient keyword** Some fields should never be saved, such as a password or a value that only makes sense in this run. Mark them transient . The JVM skips them while serializing and gives them their default value (0, false or null) after deserialization.
- **Object graphs** When an object is serialized, every object it refers to is serialized too, and every object those refer to, and so on. This set of objects is called the object graph . Every object in the graph must be Serializable.
- **Customised serialization** Marking a password transient loses it completely. If you want to save it in a protected form, add two private methods with these exact signatures. The JVM calls them automatically during serialization and deserialization.
- **Serialization and inheritance** Then every child is Serializable automatically, even without writing implements Serializable . This is why many library classes are Serializable: a parent such as Number already is.
- **Externalizable** Serializable saves the whole object automatically. Externalizable hands complete control to you: you decide which fields to save and how. It extends Serializable and adds two methods.
- **serialVersionUID** Every Serializable class has a version number called serialVersionUID . It is written into the bytes with each object. When the bytes are read, the JVM compares that number with the one in the class it has now.
- **Modern advice** For data shared with other programs or other languages, most teams prefer text formats such as JSON or compact formats such as Protocol Buffers.

FORMULAS

The two callback methods

```
private void writeObject(ObjectOutputStream out)
              throws IOException
  private void readObject(ObjectInputStream in)
    throws IOException, ClassNotFoundException
```

Lesson: Customised serialization

TABLES

Table: declaration · value after deserialization

Declaration	Value after deserialization
`int a = 10`	10 (saved and restored)
`transient int b = 20`	0 (skipped, default value)
`transient static int c = 30`	30 (static, comes from the class)
`transient final int d = 40`	40 (constant, compiled into the class)

Lesson: The transient keyword

Table: serializable · externalizable

Serializable	Externalizable
JVM saves everything automatically	you write and read each field yourself
marker interface, no methods	two methods: `writeExternal`, `readExternal`
no constructor needed in the class	public no-arg constructor is required
`transient` is meaningful	`transient` does nothing, you choose fields anyway
simple, good default	more work, best when only part of a large object matters

Lesson: Externalizable

TIPS AND TRAPS

REMEMBER Only objects of classes that implement `Serializable` (directly or through a parent) can be serialized. Anything else throws `NotSerializableException` at run time.
Lesson: What serialization means

REMEMBER String, the wrapper classes, arrays and the collection classes are all Serializable already. Objects come back in exactly the order they were written.
Lesson: Writing and reading an object

IN THE EXAM The exam favourite: `transient static` and `transient final` both compile, and in both cases `transient` has no effect.
Lesson: The transient keyword

REMEMBER Whatever `writeObject` writes, `readObject` must read back in the same order and with the same types.
Lesson: Customised serialization

COMMON MISTAKE Reversing a string is only a teaching example, not real protection. Real systems never store passwords in a serialized file. They store a salted hash in a secure place.
Lesson: Customised serialization

IN THE EXAM Writing succeeds, reading fails with `InvalidClassException`. Every non-serializable parent must have an accessible no-argument constructor.
Lesson: Serialization and inheritance

REMEMBER Always declare `private static final long serialVersionUID = 1L;` in a Serializable class. Change the number only when you deliberately break compatibility with old saved data.
Lesson: serialVersionUID

REMEMBER The javac warning `-Xlint:serial` reports Serializable classes that forget to declare it.
Lesson: serialVersionUID

COMMON MISTAKE Never deserialize bytes from a source you do not trust. Crafted input can run unexpected code during loading. Since Java 9, an `ObjectInputFilter` can limit which classes are accepted.
Lesson: Modern advice

STEP 8 JVM and Tooling

8.1 Garbage Collection

In C you must free every block of memory yourself. In Java the garbage collector does it for you. Your job is smaller but still important: stop pointing at objects you no longer need, so the collector is allowed to reclaim them.

KEY IDEAS

- **Why a garbage collector** Every object you create with new lives in an area of memory called the heap . Memory is limited, so space taken by objects nobody uses any more must be given back.
- **Four ways to make an object eligible** To watch an object without keeping it alive, these examples use a WeakReference . Its get() method returns the object while it exists, and null after the collector has removed it.
- **Requesting a collection** You can ask the JVM to run the garbage collector in two ways. Both do the same thing.
- **Looking at memory with Runtime** This program was started with java -Xmx64m Main , so the maximum heap is 64 MB. One million int values take 1,000,000 × 4 bytes ≈ 4 MB .

- **finalize and Cleaner** Early Java let a class override finalize() , a method the collector would call once before removing an object. It is now deprecated for removal (since Java 18), because it is slow, unpredictable and can bring dead objects back to life.
- **Modern collectors and good habits** The JVM ships with several collectors. You choose one with a command line flag. Their behaviour depends on the JVM vendor and version, which is why the language rules only promise when an object becomes eligible , not when it is collected.

FORMULAS

Two ways to ask

```
System.gc();                    // static method, the usual way
Runtime.getRuntime().gc();      // through the Runtime object
```

Lesson: Requesting a collection

Looking at memory with Runtime

Exam favourite

$$\text{used memory} = \text{totalMemory} - \text{freeMemory}$$

Lesson: Looking at memory with Runtime

Looking at memory with Runtime

$$1{,}000{,}000 \times 4 \text{ bytes} \approx 4 \text{ MB}$$

Lesson: Looking at memory with Runtime

Asking for 800 MB inside a 32 MB heap

```
class Main {
    public static void main(String[] args) {
        long[] huge = new long[100_000_000];
        // 100 million × 8 bytes
        System.out.println(huge.length);
    }
}
```

Lesson: Looking at memory with Runtime

The old way, do not use in new code

```
@Override
protected void finalize() throws Throwable {
    System.out.println("cleaning up");
}
```

Lesson: finalize and Cleaner

TABLES

Table: method · returns

Method	Returns
`maxMemory()`	the most heap the JVM will ever try to use, in bytes (set by `-Xmx`)
`totalMemory()`	heap currently reserved from the operating system
`freeMemory()`	the part of totalMemory not used by objects right now
`availableProcessors()`	number of processors the JVM may use

Lesson: Looking at memory with Runtime

Table: collector · flag · good for

Collector	Flag	Good for
G1 (default since Java 9)	`-XX:+UseG1GC`	most applications, balanced pauses
ZGC	`-XX:+UseZGC`	very large heaps with pauses of about a millisecond
Parallel	`-XX:+UseParallelGC`	batch jobs that care about total throughput
Serial	`-XX:+UseSerialGC`	small heaps and single processor machines

Lesson: Modern collectors and good habits

TIPS AND TRAPS

REMEMBER — An object is eligible for GC when it has no reachable reference. Being eligible does not mean it is removed at once. The JVM decides when to collect.

Lesson: Why a garbage collector

IN THE EXAM — A cycle of references does not keep objects alive. If none of them is reachable from outside, all of them are eligible.

Lesson: Four ways to make an object eligible

REMEMBER — Well written programs almost never call `System.gc()`. The JVM usually knows better when to collect. It is useful in small experiments like the ones on this page.

Lesson: Requesting a collection

COMMON MISTAKE — It is only a request. The JVM may run a collection, delay it or ignore it. A JVM started with `-XX:+DisableExplicitGC` ignores every such call.

Lesson: Requesting a collection

REMEMBER

$$\text{used memory} = \text{totalMemory} - \text{freeMemory}$$

Lesson: Looking at memory with Runtime

COMMON MISTAKE — The cleanup action must not refer to the object it watches. If it did, the object would stay reachable and never be cleaned.

Lesson: finalize and Cleaner

IN THE EXAM — Classic facts that still appear in exams: the collector calls finalize at most once per object, any exception thrown by finalize during collection is ignored, and calling `obj.finalize()` yourself is just a normal method call that does not free memory.

Lesson: finalize and Cleaner

8.2 Assertions

An assertion is a check you leave in the code to catch your own mistakes during development and testing. It costs nothing in production, because assertions are switched off unless you turn them on.

KEY IDEAS

- **Why assertions** A quick way to debug is to add System.out.println lines and remove them later. Forget one, and it prints in front of real users. Assertions solve this. You write a check once and leave it in the code. During development you run with assertions enabled .
- **assert as a keyword** Before Java 1.4, assert was an ordinary identifier, so old code could name a variable or method assert . Since 1.4 it is a reserved keyword, and such code no longer compiles.
- **The two forms** If assertions are enabled and the condition is false, the JVM throws AssertionError . If the condition is true, nothing happens.
- **Enabling and disabling** Assertions are controlled when you start the program, not when you compile it.
- **Where to use them and where not** The author believed turns % 4 is always 0 to 3. The assertion proves otherwise: in Java, the remainder of a negative number is negative. Math.floorMod(turns, 4) is the fix.
- **AssertionError** AssertionError is a child of Error , so it is unchecked. Catching it is legal, but it is bad practice: an assertion failure means the program is in a state its author believed impossible.

FORMULAS

assert cannot be a variable name

```
class Main {
public static void main(String[] args) {
int assert = 10;
System.out.println(assert);
}
}
```

Lesson: assert as a keyword

Run with java -ea Main

```
class Main {
public static void main(String[] args) {
int marks = 120;
assert marks ⩽ 100;
System.out.println("marks: " + marks);
}
}
```

Lesson: The two forms

Without -ea the list never shrinks

```
import java.util.*;
class Main {
public static void main(String[] args) {
List<String> queue = new ArrayList<>
(List.of("a", "b"));
assert queue.remove("a");
// wrong: real work inside assert
System.out.println(queue);
}
}
```

Lesson: Where to use them and where not

TABLES

Table: flag · long form · effect

Flag	Long form	Effect
`-ea`	`-enableassertions`	enable in all your classes
`-da`	`-disableassertions`	disable in all your classes (the default)
`-esa`	`-enablesystemassertions`	enable inside Java's own library classes
`-dsa`	`-disablesystemassertions`	disable inside Java's library classes

Lesson: Enabling and disabling

Table: situation · use assert · why

Situation	Use assert	Why
Arguments of a **public** method	No	Callers outside your control can pass anything. Validation must always run, so throw `IllegalArgumentException`.
Arguments of a **private** method	Yes	Only your own code calls it, so a bad value means your bug.
Command line arguments	No	They come from the user, so they need real checks.
A `default` case that should never happen	Yes	`assert false` documents and checks the impossible case.
Code with side effects in the condition	No	The program would behave differently when assertions are off.

Lesson: Where to use them and where not

TIPS AND TRAPS

IN THE EXAM

Old exam questions ask what happens when you compile this with very old `-source` settings. Modern compilers no longer accept source levels that old, so in practice `assert` is always a keyword.

Lesson: assert as a keyword

REMEMBER The second expression is evaluated **only** when the condition is false.

Lesson: The two forms

REMEMBER Flags are applied from left to right, and a more specific flag wins over a general one.

Lesson: Enabling and disabling

REMEMBER This is the only common case where a side effect inside `assert` is acceptable, because its purpose is to detect the setting itself.

Lesson: Enabling and disabling

COMMON MISTAKE With `-ea` this prints `[b]`. Without it, `[a, b]`. Never put work the program needs inside an assertion.

Lesson: Where to use them and where not

8.3 Development Tools

Writing Java is only half the job. You also need to compile it, organise the class files, package them, and run them with the right settings. This chapter walks through every command with real output.

KEY IDEAS

- **Compiling with javac** javac is the Java compiler. It reads .java source files and writes .class files containing bytecode, which any JVM can run.
- **Running with java** The java command starts a JVM, loads the class you name, and calls its main method. A class in a package must be given by its fully qualified name .
- **The classpath** The classpath is the list of places where javac and java look for classes: folders and JAR files. It can be set in three ways.
- **JAR files** A JAR (Java ARchive) packs many class files and resources into one file. It uses the zip format. Libraries are shared as JARs, and a whole application can be one JAR.
- **System properties** A system property is a name and value pair available to the whole program. The JVM sets many of them, and you can add your own with -D on the command line, before the class name.
- **JDK, JRE and JVM** JDK = JRE + development tools and JRE = JVM + library classes
- **Modern shortcuts** For a small program you can skip javac. The launcher compiles the file in memory and runs it.

FORMULAS

Some properties the JVM sets

```
class Main {
    public static void main(String[] args) {
        System.out.println("Java " +
System.getProperty("java.version").split("\\.")
                [0]);
        System.out.println("OS " +
    System.getProperty("os.name"));
        System.out.println("separator " +
System.getProperty("file.separator"));
    }
}
```

Lesson: System properties

JDK, JRE and JVM Exam favourite

$$\text{JDK} = \text{JRE} + \text{development tools}$$

Lesson: JDK, JRE and JVM

JDK, JRE and JVM

$$\text{JRE} = \text{JVM} + \text{library classes}$$

Lesson: JDK, JRE and JVM

TABLES

Table: command · meaning

Command	Meaning
`javac A.java`	compile one file, class file goes next to it
`javac A.java B.java`	compile several files
`javac *.java`	compile every .java file in the folder
`javac -d out A.java`	put class files under `out`, in package folders
`javac -cp lib.jar A.java`	compile against classes found in lib.jar
`javac --release 17 A.java`	produce class files that run on Java 17 and later

Lesson: Compiling with javac

Table: way · example · scope

Way	Example	Scope
Command line option	`java -cp out com.shop.App`	this one command only (recommended)
Environment variable	`export CLASSPATH=out`	every command in that terminal
Nothing set	`java App`	the current folder only

Lesson: The classpath

Table: command · letters mean · does

Command	Letters mean	Does
`jar -cvf shop.jar -C out .`	create, verbose, file	creates shop.jar from everything inside out
`jar -tf shop.jar`	table of contents, file	lists the entries
`jar -xvf shop.jar`	extract, verbose, file	unpacks into the current folder
`jar -cvfm app.jar manifest.txt -C out .`	create with a manifest	adds your manifest lines

Lesson: JAR files

Table: property · holds

Property	Holds
`java.version`	the running Java version
`os.name`	the operating system
`user.home`	the user's home folder
`user.dir`	the folder the program was started from
`file.separator`	`/` or `\`
`line.separator`	the line break for this system

Lesson: System properties

Table: · path · classpath

	PATH	CLASSPATH
Used by	the operating system	javac and the JVM
Finds	programs such as `javac` and `java`	class files and JAR files
Typical value	the JDK's `bin` folder	folders and JARs with your classes
Wrong value gives	"javac: command not found"	ClassNotFoundException or "package does not exist"

Lesson: JDK, JRE and JVM

TIPS AND TRAPS

REMEMBER javac works with **file names**, so you write `App.java`. The java launcher works with **class names**, so you write `com.shop.App`.
Lesson: Compiling with javac

REMEMBER An argument containing spaces needs quotes: `java Main "New Delhi"` passes one argument, not two.
Lesson: Running with java

REMEMBER Entries are separated by `:` on Linux and macOS, and by `;` on Windows. A single dot `.` means the current folder. Once you give `-cp`, the current folder is no longer included unless you add the dot.
Lesson: The classpath

IN THE EXAM The four cases above are a favourite exam pattern: which command compiles, which runs, and which error each mistake produces.
Lesson: The classpath

REMEMBER When you use `java -jar`, the `-cp` option and the CLASSPATH variable are ignored. Extra libraries must be listed in the manifest's `Class-Path` line.
Lesson: JAR files

COMMON MISTAKE The manifest file must end with a newline. Without it, the last line, often the Main-Class line, is silently ignored.
Lesson: JAR files

IN THE EXAM `-D` options go **before** the class name. Anything after the class name becomes a command line argument in `args` instead.
Lesson: System properties

REMEMBER `JAVA_HOME` is a third variable that many tools such as Maven and Gradle read. It points to the JDK folder itself, not to its bin folder.
Lesson: JDK, JRE and JVM

REMEMBER Real projects use a **build tool** such as Maven or Gradle. It downloads libraries, sets the classpath, compiles, tests and builds the JAR with one command. The commands on this page are what those tools run underneath.
Lesson: Modern shortcuts

STEP 9 Modern Java

9.1 Lambdas, functional interfaces and method references

Lambdas let you hand a small piece of behaviour to another method: how to compare, what to keep, what to do with each item. This chapter goes from your first arrow to functional interfaces, method references, composing functions, captured variables, checked exceptions and the invokedynamic machinery the JVM uses to run them.

KEY IDEAS

- **Why lambdas exist** Before Java 8, passing a small piece of behaviour (how to compare, what to do on a click, which items to keep) meant writing a whole anonymous class. Most of those lines were ceremony.
- **Lambda syntax** A lambda is (parameters) -> body . The compiler works out the parameter types from the context, so you rarely write them.
- **Functional interfaces** A functional interface is an interface with exactly one abstract method . A lambda is simply a short way to create an object of such an interface: the lambda body becomes that method.

- **Method references** When a lambda only calls one existing method, a method reference says the same thing even shorter, with :: . s -> s.toUpperCase() becomes String::toUpperCase .
- **Composing functions** Functional interfaces come with default methods that join small pieces into bigger ones, like Lego bricks. Predicate has and , or and negate . Function has andThen and compose . Comparator has comparing , thenComparing and reversed .
- **Capturing variables and this** A lambda can read local variables from the code around it, but only if they are effectively final : assigned once and never changed.
- **Lambdas and checked exceptions** The methods of Function , Consumer and friends do not declare throws , so a lambda passed to them cannot throw a checked exception such as IOException .
- **How a lambda runs inside the JVM** An anonymous class becomes its own .class file (Main$1.class). A lambda does not. javac turns its body into a private method and leaves a single invokedynamic instruction where the lambda was. The first time that line runs, the JVM builds the object.
- **Real world scenario: a retry helper** The helper does not know what it is retrying. The work arrives as a Supplier , and the meaning of "success" arrives as a Predicate . The same helper now retries API calls, database checks or UI conditions, and it reports every attempt.

FORMULAS

@FunctionalInterface catches a second abstract method

```
@FunctionalInterface
interface Broken {
    void first();
    void second();
}

class Main {
public static void main(String[] args) { }
}
```

Lesson: Functional interfaces

What javac really produces for a lambda

```
import java.util.function.Function;

class Main {
public static void main(String[] args) {
    Function<String, Integer> len = s →
        s.length();
  System.out.println(len.apply("lambda"));
    }
}
```

Lesson: How a lambda runs inside the JVM

TABLES

Table: form · example · when

Form	Example	When
No input	`() -> System.out.println("hi")`	Runnable, Supplier
One input	`s -> s.length()`	brackets are optional
Several inputs	`(a, b) -> a + b`	brackets are required
Block body	`x -> { int y = x * 2; return y; }`	more than one statement, needs return
Typed or var	`(var s) -> s.isBlank()`	to add annotations such as @NonNull

Lesson: Lambda syntax

Table: interface · method · shape · typical use in tests

Interface	Method	Shape	Typical use in tests
`Predicate<T>`	`test`	T to boolean	filter results, wait conditions
`Function<T,R>`	`apply`	T to R	map a response to a field
`Consumer<T>`	`accept`	T to nothing	log, click, fill each item
`Supplier<T>`	`get`	nothing to T	create test data lazily
`UnaryOperator<T>` `BinaryOperator<T>`	`apply`	T to T, (T,T) to T	clean up a value, add two totals
`BiFunction` `BiConsumer` `BiPredicate`	two inputs	(T,U) to ...	Map.forEach((k, v) -> ...)
`IntPredicate` `ToIntFunction` `IntSupplier` ...	primitive versions	no boxing	fast number work

Lesson: Functional interfaces

Table: kind · method reference · same as the lambda

Kind	Method reference	Same as the lambda
Static method	`Integer::parseInt`	`s -> Integer.parseInt(s)`
Method of a given object	`System.out::println`	`x -> System.out.println(x)`
Method of the incoming object	`String::length`	`s -> s.length()`
Constructor	`ArrayList::new`	`() -> new ArrayList<>()`

Lesson: Method references

TIPS AND TRAPS

REMEMBER Selenium's `wait.until(driver -> driver.findElement(By.id("total")).isDisplayed())` takes a `Function<WebDriver, T>`. Now you know why a lambda fits there.

Lesson: Functional interfaces

COMMON MISTAKE Do not hide exceptions with an empty catch inside a lambda. A test that swallows an IOException will pass with wrong data, which is worse than a clear failure.

Lesson: Lambdas and checked exceptions

9.2 Streams, Collectors, Optional and Gatherers

Streams let you describe what you want from a collection: keep these, change them to that, group by this, stop after three. This chapter covers lazy pipelines, every common operation, collectors for reports, primitive streams, Optional, the stream gatherers that became final in Java 24, and parallel streams with their traps.

KEY IDEAS

- **What a stream is** A stream is a pipeline that data flows through. You describe the steps (keep these, change them to that, stop after three), and Java runs them for you. A stream does not store data.
- **Laziness and short circuits** Intermediate operations are lazy . Building the pipeline does nothing. When the terminal operation asks for a result, elements travel through all the steps one at a time , and the work stops as soon as the answer is known.
- **Creating streams** Almost anything can feed a stream: collections, arrays, number ranges, single values, generators, the characters of a string and the lines of a file (Files.lines , covered in the file chapter).
- **Intermediate operations**
- **Terminal operations** A terminal operation runs the pipeline and returns something that is not a stream: a number, a boolean, an Optional, a collection, or nothing at all (forEach).
- **Collectors: grouping and summarising** collect with the helpers in Collectors answers report style questions in one line: how many per status, average time per suite, names of the slow tests.

- **Primitive streams** Stream<Integer> stores every number as an object. IntStream , LongStream and DoubleStream work with plain numbers, so they are faster and add sum , average and summaryStatistics . Use mapToInt to enter them and boxed to leave.
- **Optional** Optional<T> is a box that holds one value or nothing. A method that returns Optional tells the caller, in its signature, "I might not find anything". It replaces returning null and the NullPointerException that follows.
- **Stream gatherers (Java 24)** Before Java 24, if no built in intermediate operation fitted, you had to leave the stream. Gatherers (JEP 485, final in Java 24, so in the Java 25 LTS) are custom intermediate operations, plugged in with gather(...) . The JDK ships useful ones in Gatherers .
- **Parallel streams** parallel() splits the work across the CPU cores using the common ForkJoinPool. It helps for large amounts of pure calculation with no shared state. It hurts for small lists, for I/O, and for anything that writes to a shared object.
- **Real world scenario: a nightly test report** A test that both passed and failed across runs is flaky : groupingBy plus mapping to a Set of statuses finds them in one pass. The same few lines work on real data read from JUnit XML or a database.

FORMULAS

A stream can only be consumed once

```
import java.util.stream.*;

class Main {
public static void main(String[] args) {
Stream<String> s = Stream.of("a", "b",
"c");
System.out.println(s.count());
System.out.println(s.count());       // a
stream can be used only once
}
}
```

Lesson: Laziness and short circuits

TABLES

Table: operation · what it does

Operation	What it does
`filter`	keeps elements that pass a Predicate
`map` `mapToInt`	turns each element into another value
`flatMap` `mapMulti`	turns each element into zero or more elements and flattens them
`distinct` `sorted`	removes duplicates (equals), sorts (stateful, they must see everything)
`limit` `skip`	first n, or all but the first n
`takeWhile` `dropWhile`	Java 9: until a condition fails, or from where it fails
`peek`	looks at each element without changing it, for debugging only

Lesson: Intermediate operations

Table: question · collector

Question	Collector
How many in each group	`groupingBy(key, counting())`
Split into yes and no	`partitioningBy(predicate)`
One text from many	`joining(", ")`
Look up by key	`toMap(key, value, merge)`
Min, max, average, sum at once	`summarizingInt`
Two answers combined	`teeing(a, b, combine)`

Lesson: Collectors: grouping and summarising

Table: do · do not

Do	Do not
Return Optional from a find or lookup method	Use Optional for fields or method parameters
Chain `map`, `filter`, `orElse`	Call `get()` without checking
Use `orElseThrow` with a clear message	Return `null` from a method declared to return Optional

Lesson: Optional

TIPS AND TRAPS

REMEMBER Pass `TreeMap::new` to `groupingBy` when you want the groups in sorted order. A plain HashMap has no reliable order, which makes report output and test assertions unstable.

Lesson: Collectors: grouping and summarising

COMMON MISTAKE `orElse(x)` always evaluates `x`, even when a value is present. If the default is expensive (a database call, creating a user), use `orElseGet(() -> ...)`, which runs only when the box is empty.

Lesson: Optional

REMEMBER Real uses: send test data rows to an API in batches of 100 (`windowFixed`), alert when the moving average response time rises (`windowSliding`), and call a slow service for many ids with limited parallelism (`mapConcurrent`, which uses virtual threads).

Lesson: Stream gatherers (Java 24)

9.3 Records, sealed types and pattern matching

Modern Java writes far less code to say far more. This chapter covers var, text blocks for JSON and SQL, switch expressions, records, sealed classes, pattern matching for instanceof and switch, record patterns, the unnamed variable and the primitive patterns previewed in Java 27, ending with API results modelled as data.

KEY IDEAS

- **var: let the compiler write the type** var (Java 10) asks the compiler to work out the type of a local variable from the value on the right. The variable still has one fixed type. It is not like JavaScript, where a variable can hold anything.
- **Text blocks** A text block (Java 15) is a string that spans several lines between triple quotes. Indentation shared by all lines is removed, line breaks are kept, and double quotes need no escaping. It is perfect for JSON request bodies, SQL and expected HTML in tests.
- **Switch expressions** The old switch statement falls through to the next case unless you remember break . That single missing word has caused many real bugs.

- **Records** A record (Java 16) is a class whose job is to carry data. From one line, Java writes the private final fields, the constructor, an accessor per field (named name() , not getName()), and equals , hashCode and toString based on all fields.
- **Sealed classes and interfaces** A sealed type (Java 17) lists exactly which classes may extend or implement it, using permits . It is the middle ground between final (nobody) and a normal class (anybody).
- **Pattern matching for instanceof** The old pattern was test, then cast, then use. Since Java 16, o instanceof String s does all three: if the test passes, s is a String variable ready to use. The variable is only in scope where the test is known to be true, a rule called flow scoping .
- **Pattern matching for switch** Since Java 21, a switch can test the type of a value, add a condition with when (a guard), and even handle null as a case. Cases are checked from top to bottom and the first match wins.
- **Record patterns and unnamed variables** A record pattern (Java 21) matches a record and pulls its fields out into variables in the same step, and patterns can be nested. Combined with sealed types and switch, this style is called data oriented programming .
- **Primitive patterns (preview in Java 27)** Patterns so far only worked with objects. Primitive types in patterns, instanceof and switch (JEP 532, fifth preview in Java 27) lets you write case int c when c >= 500 and ask big instanceof int , which is true only when the value fits into an int without ...
- **Real world scenario: modelling API outcomes** Every possible outcome is now a record that implements one sealed interface. The report switch must handle them all, guards pick out special cases such as a zero total or a 503, and nested record patterns read the order fields directly.

FORMULAS

Where var is not allowed

```
                class Main {
      public static void main(String[] args) {
        var nothing;                      // needs
                an initial value
         var empty = null;                // null
                has no useful type
                      var x = 10;
         x = "ten";                       // the
            type is fixed once chosen
                        }
                       }
```

Lesson: var: let the compiler write the type

TIPS AND TRAPS

REMEMBER Use `var` when the type is obvious from the right side, such as `var users = new ArrayList<User>()`. Keep the type when it helps the reader, such as `int timeoutMs = config.get("timeout")`.

Lesson: var: let the compiler write the type

REMEMBER Records are ideal for API test models: request bodies, expected responses, test data rows. Jackson 2.12 and later can read JSON straight into records, and two records with the same values are equal, which makes assertions simple.

Lesson: Records

COMMON MISTAKE Preview features can still change. They need `--enable-preview` to compile and run, so keep them out of production code and use them to try ideas early.

Lesson: Primitive patterns (preview in Java 27)

9.4 Concurrency in depth: from executors to virtual threads

Doing many things at once is where Java programs get fast, and where the hardest bugs hide. This chapter builds on the multithreading chapter with executors and futures, race conditions and their fixes, the memory model, concurrent

collections, latches and semaphores, CompletableFuture, virtual threads from Java 21, and structured concurrency and scoped values from Java 25 to 27.

KEY IDEAS

- **Executors: tasks instead of threads** The multithreading chapter created threads by hand. Real programs rarely do that. They hand tasks to an ExecutorService , which owns a pool of threads and reuses them, the way a restaurant gives orders to a fixed team of cooks instead of hiring a cook per ...
- **Race conditions and four fixes** When two threads change the same data without coordination, the result depends on timing. That is a race condition . It is the most common reason for a test that fails "only sometimes on CI".
- **Visibility and the Java memory model** Each CPU core keeps its own cache, and the compiler may reorder instructions. So a value written by one thread is not guaranteed to be seen by another, unless there is a happens-before relationship between them.
- **Concurrent collections** A normal HashMap shared by threads can lose entries or, in old Java versions, even loop forever. The java.util.concurrent collections are built for sharing.
- **Synchronizers: latches and semaphores** Synchronizers coordinate threads. A CountDownLatch lets threads wait until a count reaches zero, such as "start the tests when the database, cache and API are ready".
- **CompletableFuture** Future.get() blocks. CompletableFuture (Java 8) lets you describe what to do when the result arrives: transform it, combine it with another, or recover from an error, without blocking a thread while you wait.
- **Virtual threads (Java 21)** A normal (platform) thread is a real operating system thread with about a megabyte of stack, so a server can afford a few thousand.
- **Structured concurrency and scoped values** Structured concurrency treats a group of subtasks as one unit: they start inside a block, and the block does not end until all of them are done. If one fails, the others are cancelled. No thread is left running in the background by accident.
- **Real world scenario: a fast smoke test** Each endpoint gets its own virtual thread, a Semaphore keeps the load at 50 calls at a time, and the results go into concurrent collections. The code reads like a simple loop, finishes in a few seconds, and the report is sorted so it is the same on every run.

FORMULAS

A race condition: two threads, one counter

```
                class Main {
              static int count = 0;
    public static void main(String[] args) throws
                Exception {
        Runnable work = () -> { for (int i = 0; i
          < 1_000_000; i++) count++; };
           Thread a = new Thread(work), b = new
                   Thread(work);
        a.start(); b.start(); a.join(); b.join();
           System.out.println(count);       //
                expected 2000000
                        }
                      }
```

Lesson: Race conditions and four fixes

TABLES

Table: factory · gives

Factory	Gives
`newFixedThreadPool(n)`	n threads, extra tasks wait in a queue
`newCachedThreadPool()`	creates threads as needed, reuses idle ones
`newScheduledThreadPool(n)`	run later or every few seconds
`newVirtualThreadPerTaskExecutor()`	Java 21: a new virtual thread for every task

Lesson: Executors: tasks instead of threads

Table: tool · use it when

Tool	Use it when
`synchronized`	simple, short critical sections
`AtomicInteger` `AtomicReference`	one variable, lock free, uses compare and swap (CAS)
`LongAdder`	counters hit by many threads, such as metrics
`ReentrantLock`	you need tryLock, timeouts, fairness or several conditions

Lesson: Race conditions and four fixes

Table: these create happens-before

These create happens-before
Unlocking a monitor or lock, then locking the same one
Writing a `volatile` field, then reading it
`Thread.start()`, and everything a thread did before another thread's `join()` returns
Putting into a concurrent collection, then taking it out

Lesson: Visibility and the Java memory model

Table: method · meaning

Method	Meaning
`supplyAsync` `runAsync`	start work on another thread
`thenApply` `thenCompose`	transform the value, or chain another async call
`thenCombine` `allOf` `anyOf`	join two, all or the first of several
`exceptionally` `handle` `whenComplete`	recover, see value and error, or just log
`orTimeout` `completeOnTimeout`	Java 9: fail or use a default when too slow

Lesson: CompletableFuture

TIPS AND TRAPS

COMMON MISTAKE `volatile` fixes **visibility**, not **atomicity**. `volatileCount++` still loses updates.

Lesson: Visibility and the Java memory model

REMEMBER The bug to look for in code reviews is "check then act": `if (!map.containsKey(k)) map.put(k, v)`. Between the check and the put, another thread can get in. Use `putIfAbsent`, `computeIfAbsent` or `merge`, which do both in one atomic step.

Lesson: Concurrent collections

REMEMBER

Rules for virtual threads: create one per task and never pool them, keep using plain blocking code (no callbacks needed), limit access to scarce resources with a Semaphore instead of a small pool, and do not expect speed ups for pure calculation. Since Java 24 (JEP 491), `synchronized` blocks no longer pin a virtual thread to its carrier.

Lesson: Virtual threads (Java 21)

9.5 Inside the JVM: how a Java program really runs

What happens between typing java Main and seeing output? This chapter follows a program through the class file, the class loaders, the memory areas, bytecode, the JIT compiler and the garbage collector, then shows the tools that make Java services fast and easy to diagnose: Flight Recorder, the AOT cache from Java 24 to 26, compact object headers and G1 everywhere in Java 27.

KEY IDEAS

- **The journey of a Java program** The basics chapter showed the short version: javac makes bytecode and the JVM runs it. This chapter opens the JVM and looks at each stage, with real commands you can type.
- **Inside a class file** A .class file is a small binary file with a fixed layout: a magic number, a version, a constant pool (all the names and literal values the class uses), the fields, the methods with their bytecode, and some attributes.
- **Class loading: delegation and lazy initialisation** The JVM does not load every class at start. It loads a class the first time the code needs it. Three built in class loaders share the work, and each one asks its parent first.
- **The runtime memory areas** A running JVM divides its memory into areas with different jobs. Every thread has its own stack of frames, one frame per method call, holding that call's local variables. All objects live on the shared heap .
- **Bytecode and the operand stack** Bytecode instructions are one byte long, plus any operands, which is where the name comes from. The JVM is a stack machine : instructions take their inputs from a small operand stack in the current frame and push their results back onto it.
- **The interpreter and the JIT compiler** HotSpot starts every method in the interpreter . It counts how often each method runs and how often each loop turns. When a method gets hot, the C1 compiler turns it into machine code quickly (tier 2 or tier 3, the second one also collects a profile).
- **Garbage collectors and object headers** The garbage collection chapter explained when an object becomes garbage. The JVM ships several collectors that differ in how they clean up.
- **JDK Flight Recorder: see what really happened** JDK Flight Recorder (JFR) is built into the JVM and records events such as garbage collections, slow locks, file and socket reads, and CPU samples, with an overhead of about one percent.
- **Faster startup with the AOT cache** Each time a Java program starts, the JVM loads, verifies and links the same classes again. Project Leyden fixes that with an ahead of time cache .
- **Real world scenario: a service that misbehaves in a small container** The first step is always to ask the JVM how it was started. A small health check endpoint can report it.

FORMULAS

Every class file starts with CAFEBABE

```
class Main {
    public static void main(String[] args) {
        System.out.println("Hello from
            bytecode");
    }
}
```

Lesson: Inside a class file

Java 25 picks a collector from the machine size

```
class Main {
    public static void main(String[] args) {
        System.out.println("Hello from
            bytecode");
    }
}
```

Lesson: Garbage collectors and object headers

TABLES

Table: java · class file major version

Java	Class file major version
8	52
11	55
17	61
21	65
25	69
27	71

Lesson: Inside a class file

Table: area · holds · error when full · size flag

Area	Holds	Error when full	Size flag
Stack (per thread)	frames, local variables	StackOverflowError	`-Xss`
Heap	all objects and arrays	OutOfMemoryError: Java heap space	`-Xms -Xmx`
Metaspace	classes and methods	OutOfMemoryError: Metaspace	`-XX:MaxMetaspaceSize`
Code cache	JIT machine code	a warning, then the JIT stops	`-XX:ReservedCodeCacheSize`

Lesson: The runtime memory areas

Table: collector · best for · flag

Collector	Best for	Flag
G1	the default: balanced pauses and throughput	`-XX:+UseG1GC`
ZGC	very short pauses (under a millisecond) on large heaps, generational only since Java 24	`-XX:+UseZGC`
Parallel	batch jobs that only care about throughput	`-XX:+UseParallelGC`
Serial	tiny heaps and single core tools	`-XX:+UseSerialGC`

Lesson: Garbage collectors and object headers

Table: problem · flag or tool

Problem	Flag or tool
Heap too small or too large for the container	`-XX:MaxRAMPercentage=75`
Serial collector picked on a small machine	`-XX:+UseG1GC`, or move to Java 27
Slow first start	`-XX:AOTCache=app.aot` from a training run
Too much memory for many small objects	compact object headers (default in Java 27)
Nobody knows where time goes	`jcmd <pid> JFR.start` and a custom event per report
Crash with no clues	`-XX:+HeapDumpOnOutOfMemoryError`

Lesson: Real world scenario: a service that misbehaves in a small container

TIPS AND TRAPS

REMEMBER

ClassNotFoundException means code asked for a class by name and no loader found it. NoClassDefFoundError means the class was there when the code was compiled but is missing or failed to initialise at run time. Both usually point to a missing JAR on the classpath.

Lesson: Class loading: delegation and lazy initialisation

REMEMBER Never measure Java speed with one run of a loop and System.currentTimeMillis. Use JMH, the Java Microbenchmark Harness, which handles warm up, dead code removal and statistics for you.

Lesson: The interpreter and the JIT compiler

REMEMBER On a live server you do not need to restart anything: `jcmd <pid> JFR.start duration=60s filename=prod.jfr` records for a minute, and JDK Mission Control or `jfr print` opens the file.

Lesson: JDK Flight Recorder: see what really happened

9.6 Annotations, reflection and modules

Frameworks such as JUnit and Spring seem to call your code by magic. This chapter shows the real mechanism: annotations as labels, reflection to read them, and proxies to wrap objects. Then it covers the module system that protects code from misuse, jlink for small runtimes, module imports from Java 25, and the Java 26 warning about changing final fields.

KEY IDEAS

- **Annotations: labels the compiler and tools can read** An annotation is a label that starts with @ . It does not change what your code does by itself. The compiler, build tools or frameworks read the label and act on it, like a "fragile" sticker on a parcel that tells the courier how to handle it.
- **Writing your own annotation** You declare an annotation with @interface . Two meta annotations decide how it behaves. @Retention says how long it lives: only in the source (SOURCE), in the class file (CLASS, the default), or also at run time (RUNTIME).
- **Reflection: looking at classes while the program runs** Reflection lets code examine and use classes it did not know about when it was compiled. Every loaded class has a Class object that lists its fields, methods, constructors and, for records, its components.
- **Dynamic proxies: how mocks and frameworks wrap your objects** A dynamic proxy is an object the JVM creates at run time that implements one or more interfaces. Every call on it goes to one handler method, which can log, time, check permissions or forward the call.
- **The module system (Java 9)** Before Java 9, a public class in any JAR could be used by any other JAR. Big programs ended up depending on each other's internals. A module fixes this.
- **Module import declarations (Java 25)** A long list of imports at the top of a file is noise, especially for beginners and small scripts. Java 25 made module import declarations final (JEP 511).
- **Real world scenario: validating a sign up form** The rules now live next to the data as annotations, and one generic method checks any record. This is the same idea as Jakarta Bean Validation (@NotBlank , @Min , @Max), which Spring runs automatically on request bodies.

FORMULAS

Strong encapsulation: the JDK keeps its private parts private

```
import java.lang.reflect.Field;

class Main {
public static void main(String[] args) throws
Exception {
Field value =
String.class.getDeclaredField("value");
System.out.println("found the private
field: " + value.getName());
value.setAccessible(true);            //
java.base does not open java.lang to us
System.out.println("never printed");
}
}
```

Lesson: Reflection: looking at classes while the program runs

Two modules: a core library and an app

```
module com.shop.core {
exports com.shop.core.api;      // other
modules may use this package
// com.shop.core.internal is not exported, so
it stays private to this module
}
```

Lesson: The module system (Java 9)

Two modules export a class with the same name

```
import module java.sql;

void main() {
Date today = new Date(0);       //
java.util.Date or java.sql.Date?
System.out.println(today);
}
```

Lesson: Module import declarations (Java 25)

TABLES

Table: annotation · what it tells the compiler

Annotation	What it tells the compiler
`@Override`	this must override or implement a method, or fail
`@Deprecated(since, forRemoval)`	warn everyone who still uses this
`@SuppressWarnings("...")`	I know, stop warning me about this here
`@FunctionalInterface`	this interface must have exactly one abstract method
`@SafeVarargs`	this generic varargs method does not misuse its array

Lesson: Annotations: labels the compiler and tools can read

Table: module-info line · meaning

module-info line	Meaning
`requires m;`	I use module m
`requires transitive m;`	I use m, and so does anyone who requires me
`exports p;`	other modules may use package p at compile and run time
`opens p;`	frameworks may use deep reflection on p, for example JSON mapping
`uses S;` and `provides S with Impl;`	plug in implementations with ServiceLoader

Lesson: The module system (Java 9)

TIPS AND TRAPS

REMEMBER java.lang.reflect.Proxy only works with interfaces. Libraries such as Mockito and Spring generate subclasses with bytecode tools when they need to wrap plain classes, which is one reason the Class-File API (final in Java 24) was added to the JDK.

Lesson: Dynamic proxies: how mocks and frameworks wrap your objects

9.7 Databases and web APIs: JDBC and HttpClient

Programs become useful when they talk to the outside world. This chapter connects Java to a real database with JDBC, shows SQL injection and how PreparedStatement stops it, groups changes into transactions, and then calls web APIs with HttpClient, including timeouts, async calls, HTTP/3 from Java 26 and a small API test suite.

KEY IDEAS

- **JDBC: talking to a database** Almost every real application stores data in a database. JDBC (Java Database Connectivity) is the standard API in java.sql for that. Your code uses the same interfaces whatever the database, and a driver JAR does the database specific work.
- **PreparedStatement and SQL injection** The most famous database bug is SQL injection . When user input is glued into SQL text, a quote in the input can end the string early and change the meaning of the query. Try it in the widget first, then see the real database do it.

- **Transactions: all or nothing** A bank transfer is two updates: take money from one account and add it to another. If the program crashes between them, money appears or disappears. A transaction groups statements so they all succeed together (commit) or all are undone (rollback).
- **Generated keys, batches and records** Three everyday needs: getting the id the database generated for a new row, inserting many rows quickly, and turning rows into Java objects. Records are a natural fit for rows.
- **HttpClient: calling web APIs (Java 11)** Since Java 11, the JDK has a modern HTTP client in java.net.http . It supports HTTP/1.1 and HTTP/2, synchronous and asynchronous calls, and timeouts. Each example below starts a tiny local server first with com.sun.net.httpserver , so the calls really happen.
- **Timeouts, async calls and HTTP/3** A call to another service can hang. Always set a connect timeout on the client and a request timeout on each request. Since Java 26 the request timeout also covers reading the response body, not only waiting for the headers.
- **Real world scenario: an API test suite with only the JDK** The one failing check is there on purpose, to show what a failure report looks like. In bigger projects the same checks are written with REST Assured or JUnit, but they still send these requests and compare these status codes, headers and bodies.

TABLES

Table: acid · means

ACID	Means
Atomic	all statements or none
Consistent	rules such as CHECK constraints always hold
Isolated	other users do not see half finished work
Durable	after commit, the data survives a crash

Lesson: Transactions: all or nothing

Table: status · meaning

Status	Meaning
200 OK, 201 Created, 204 No Content	it worked
301, 302, 304	look elsewhere, or use your cached copy
400, 401, 403, 404, 405, 409, 429	the client did something wrong
500, 502, 503, 504	the server failed

Lesson: HttpClient: calling web APIs (Java 11)

TIPS AND TRAPS

REMEMBER Every Connection, Statement and ResultSet holds resources in the database. Always open them in try-with-resources so they close even when an exception is thrown. Leaked connections are a classic cause of "the app froze after a few hours".

Lesson: JDBC: talking to a database

REMEMBER In real projects, libraries such as Spring's JdbcClient, jOOQ or JPA with Hibernate remove most of this code. They all use JDBC underneath, so knowing it helps when you need to read their logs or fix a slow query.

Lesson: Generated keys, batches and records

9.8 What is new in Java: from Java 21 to Java 27

Java moves fast: a release every six months and a long term support version every two years. This chapter walks through what changed from the Java 21 LTS to Java 27, runs every feature on a real JDK, and explains where each one helps at work, from sequenced collections and native calls to quantum safe cryptography and simpler programs.

KEY IDEAS

- **The release train, LTS and preview features** Java ships a new version every six months, in March and September. Java 25 (September 2025) is the latest long term support (LTS) release, Java 26 came in March 2026 and Java 27 in September 2026.
- **Java 21 LTS: sequenced collections** Java 21 brought virtual threads, record patterns and pattern matching for switch, each covered in its own chapter. One small change you will use daily is sequenced collections .
- **Java 22 and 23: native code, multi file launch, Markdown docs** The foreign function and memory API (JEP 454, final in Java 22) lets Java call C libraries and work with memory outside the heap safely, replacing most JNI code. Database drivers, machine learning libraries and image tools use it to reach native code.
- **Java 24: quantum safe cryptography** Big quantum computers could one day break RSA and elliptic curve keys. Attackers can record encrypted traffic today and decrypt it later.
- **Java 25 LTS: simpler programs and safer constructors** Java 25 made starting with Java much simpler. Compact source files and instance main methods (JEP 512) remove the class, static and String[] args ceremony, and the new IO class reads and prints text without System.out or a Scanner.
- **Java 26 and 27: what is final and what to try** Java 26 added HTTP/3 to HttpClient (JDBC and HTTP chapter), warns before final fields can be changed by reflection (modules chapter), removed the old Applet API (JEP 504) and let the AOT cache work with every garbage collector (JVM chapter).
- **Real world scenario: upgrading a service from Java 17 to Java 25** Step one is finding code that relies on JDK internals, because strong encapsulation and the removal of old APIs are what usually break upgrades.

FORMULAS

Run a small program of several files without javac (Java 22)

```
void main() {
IO.println(Prices.withTax(200));
}
```

Lesson: Java 22 and 23: native code, multi file launch, Markdown docs

Compact source files, instance main and IO (Java 25)

```
void main() {
  String name = IO.readln("Your name: ");
IO.println("Hello, " + name + "! This is the
        whole program.");
}
```

Lesson: Java 25 LTS: simpler programs and safer constructors

TABLES

Table: still in preview or incubator in java 27 · round

Still in preview or incubator in Java 27	Round
Structured concurrency (JEP 533)	7th preview
Primitive types in patterns (JEP 532)	5th preview
Lazy constants (JEP 531)	3rd preview
PEM encodings (JEP 538)	3rd preview
Vector API (JEP 537)	12th incubator

Lesson: Java 26 and 27: what is final and what to try

Table: step · why

Step	Why
Run `jdeps --jdk-internals` and `jdeprscan`	find internal and deprecated API use early
Upgrade build plugins, Lombok, Mockito, ByteBuddy, Spring	libraries that generate bytecode must know the new class file version
Build and test with the new JDK in CI first	catch behaviour changes before production
Read the release notes for removed options	for example the Security Manager can no longer be enabled since Java 24
Then adopt features one by one	virtual threads for blocking I/O, records for DTOs, pattern matching for parsing

Lesson: Real world scenario: upgrading a service from Java 17 to Java 25